AZURE

CHEAT SHEETS

Microsoft Azure for developers and operators, from az CLI to Foundry

az CLI · Subscriptions · Entra ID · RBAC · Managed Identity · Key Vault · Azure Policy · VNets · Private Endpoints · NSGs · Front Door · VMs · Scale Sets · Functions · App Service · API Management · Service Bus · Event Grid · ACR · Container Apps · AKS · Blob Storage · Azure SQL · Cosmos DB · Bicep · azd · Terraform · Azure Monitor · KQL · Microsoft Foundry

42 REFERENCE SHEETS

CONTENTS

GETTING STARTED

01 Overview & Global Infrastructure
02 Subscriptions & Billing
03 Azure CLI
04 SDKs & Credentials

IDENTITY & SECURITY

05 Microsoft Entra ID
06 Azure RBAC
07 Managed Identities & Federation
08 Key Vault
09 Security & Governance

NETWORKING

10 Virtual Networks
11 NAT, Private Endpoints & Connectivity
12 NSGs & Azure Firewall
13 Azure DNS & Traffic Manager
14 Front Door & Certificates

COMPUTE

15 Virtual Machines
16 VM Operations & Pricing
17 Load Balancing
18 Virtual Machine Scale Sets

SERVERLESS & MESSAGING

20 Azure Functions Basics
21 App Service & Functions Operations
22 API Management
23 Event Grid & Logic Apps
24 Service Bus & Storage Queues

CONTAINERS

25 Azure Container Registry
26 Azure Container Apps
27 AKS

STORAGE

28 Blob Storage Basics
29 Storage Security & Features
30 Disks, Files & Backup

DATABASES

31 Azure SQL & PostgreSQL
32 Cosmos DB
33 Redis & Other Data Services

INFRASTRUCTURE AS CODE

34 Bicep
35 Azure Developer CLI (azd)
36 Terraform on Azure

OBSERVABILITY & DEPLOYMENT

37 Azure Monitor Metrics & Alerts
38 Log Analytics & Application Insights
39 Deploying to Azure

AI & MACHINE LEARNING

40 Microsoft Foundry & Azure OpenAI
41 Foundry Tools & ML

QUICK REFERENCE

42 Quick Reference — az Commands & Service Picker

AZ

Table of Contents

42 sheets · syntax · examples · notes

GETTING STARTED 01 – 04

- 01 Overview & Global Infr…
- 02 Subscriptions & Billing
- 03 Azure CLI
- 04 SDKs & Credentials

IDENTITY & SECURITY 05 – 09

- 05 Microsoft Entra ID
- 06 Azure RBAC
- 07 Managed Identities & F…
- 08 Key Vault
- 09 Security & Governance

NETWORKING 10 – 14

- 10 Virtual Networks
- 11 NAT, Private Endpoints …
- 12 NSGs & Azure Firewall
- 13 Azure DNS & Traffic Ma…
- 14 Front Door & Certificat…

COMPUTE 15 – 19

- 15 Virtual Machines
- 16 VM Operations & Prici…
- 17 Load Balancing
- 18 Virtual Machine Scale …
- 19 Choosing Compute

SERVERLESS & MESSAGING 20 – 24

- 20 Azure Functions Basics
- 21 App Service & Functio…
- 22 API Management
- 23 Event Grid & Logic Apps
- 24 Service Bus & Storage …

CONTAINERS 25 – 27

- 25 Azure Container Regist…
- 26 Azure Container Apps
- 27 AKS

STORAGE 28 – 30

- 28 Blob Storage Basics
- 29 Storage Security & Fea…
- 30 Disks, Files & Backup

DATABASES 31 – 33

- 31 Azure SQL & PostgreS…
- 32 Cosmos DB
- 33 Redis & Other Data Ser…

INFRASTRUCTURE AS CODE 34 – 36

- 34 Bicep
- 35 Azure Developer CLI (a…
- 36 Terraform on Azure

OBSERVABILITY & DEPLOYMENT 37 – 39

- 37 Azure Monitor Metrics …
- 38 Log Analytics & Applic…
- 39 Deploying to Azure

AI & MACHINE LEARNING 40 – 41

- 40 Microsoft Foundry & A…
- 41 Foundry Tools & ML

QUICK REFERENCE 42

- 42 Quick Reference — az …

Sheet 01

Overview & Global Infrastructure

regions · availability zones · region pairs · tenants, management groups, subscriptions & resource groups · Azure Resource Manager · resource IDs

MICROSOFT'S CLOUD

Azure runs in 60+ **regions**; most large regions have three **availability zones**, and many are matched with a **paired region** for platform updates and recovery. Every request goes through **Azure Resource Manager (ARM)**, which places each resource in a **resource group** inside a **subscription**, inside a Microsoft Entra **tenant**. Management groups sit above subscriptions to apply policy at scale.

KEY POINTS

- Region names: `eastus2`, `westeurope`, `centralus`
- Zones are numbered 1, 2, 3; numbering is mapped per subscription
- A resource group is a lifecycle boundary: delete it and everything inside goes
- Every resource has a full resource ID used by RBAC, CLI and IaC

EXPLORE THE HIERARCHY

```
az login
az account tenant list -o table              # tenants you can reach
az account management-group list -o table    # needs permission at root
az account list -o table                     # subscriptions
az account list-locations \
    --query "[].{name:name, geo:metadata.geographyGroup}" \
    -o table

az group create -n rg-shop-dev -l eastus2 --tags owner=team-a env=dev
az resource list -g rg-shop-dev -o table

# Every resource has an ID like this
# /subscriptions/<sub-id>/resourceGroups/rg-shop-dev/providers/
#   Microsoft.Storage/storageAccounts/shopdevdata
az resource show --ids <resource-id> --query location
```

CODE EXPLAINED

- → `az login` opens a browser; `--use-device-code` works over SSH
- → Locations list shows the geography each region belongs to
- → Resource groups hold resources that share a lifecycle
- → Resource IDs name provider, type and resource in one path

AWS / GCP EQUIVALENT

- ↔ **AWS** Regions, AZs, Organizations / accounts (aws-01)
- ↔ **GCP** Regions, zones, organization / folders / projects (gcp-01)
- ↔ Azure adds the resource group layer: no direct AWS or GCP twin

COST CHECK

- $ Free account: credit for the first 30 days plus 12 months of select free services
- $ Region choice changes price; compare in the pricing calculator
- $ Zone-redundant SKUs often cost more than single-zone ones

HIERARCHY

LEVEL	HOLDS	USED FOR
Tenant (Entra ID)	Identities	Sign-in, users, apps
Management group	Subscriptions	Policy and RBAC at scale
Subscription	Resource groups	Billing and quota boundary
Resource group	Resources	Lifecycle and access
Resource	—	A VM, database, vault…

WAYS TO WORK

TOOL	USE
Portal	Explore, one-off tasks
Azure CLI (az)	Scripts, this set
Azure PowerShell	Windows-heavy shops
Bicep / Terraform	Repeatable infrastructure
SDKs	Apps and automation

DO & DON'T

- ✓ **Do** Name resource groups by app, env and region (`rg-shop-prod-eus2`)
- ✓ **Do** Tag every resource group with owner and cost center
- ✗ **Don't** Mix unrelated apps in one resource group
- ✗ **Don't** Assume a region has every service: check product availability by region

LOCAL TESTING

- ⌂ Azure Cloud Shell has `az`, Bicep, Terraform and kubectl preinstalled
- ⌂ A free account sandbox with a budget alert is safe for practice
- ⌂ Microsoft Learn sandboxes give temporary subscriptions for exercises

PRODUCTION CHECKLIST

- ☐ Management groups for prod / non-prod with policies attached
- ☐ Zone-redundant SKUs for production services
- ☐ Resource naming and tagging standards enforced by Azure Policy

Subscriptions & Billing

PAYING FOR AZURE

A **subscription** is the billing and quota container: charges roll up from resources to resource groups to subscriptions to a billing account. **Cost Management** shows spend by service, resource group or tag, **budgets** send alerts at thresholds, and **Azure Advisor** recommends savings. Tags make costs traceable to teams.

KEY POINTS

- Budgets alert; they do not stop resources (use automation for that)
- Tags are not inherited by default; use Azure Policy to inherit or require them
- Cost data refreshes several times a day, not in real time
- Separate subscriptions per environment keep bills and quotas apart

LIMITS & QUOTAS

- ≤ 980 resource groups per subscription
- ≤ 50 tags per resource; tag names up to 512 chars (128 on storage accounts)
- ≤ 800 deployments kept per resource group history (older ones auto-deleted)

BUDGETS, TAGS AND COST QUERIES

```
# Monthly budget with an email at 80%
az consumption budget create --budget-name shop-monthly \
    --amount 300 --category cost --time-grain Monthly \
    --start-date 2026-12-01 --end-date 2027-12-01 \
    --resource-group rg-shop-prod

# Tag a resource group and everything you create later
az group update -n rg-shop-prod --tags owner=team-a costcenter=1234
az tag list --resource-id /subscriptions/<sub-id>/resourceGroups/rg-shop-prod

# Usage for the last month
az consumption usage list --start-date 2026-10-01 --end-date 2026-10-31 \
    --query "[].{name:instanceName, cost:pretaxCost}" -o table

# Savings advice
az advisor recommendation list --category Cost -o table

# Quota usage per region (vCPUs and more)
az vm list-usage -l eastus2 -o table
```

CODE EXPLAINED

- → Budgets can be scoped to a subscription or resource group
- → `az group update --tags` replaces the tag set; merge with `az tag update --operation merge`
- → Usage lines map cost to individual resources
- → `list-usage` shows how close you are to vCPU quotas

AWS / GCP EQUIVALENT

- ↔ **AWS** AWS Budgets, Cost Explorer, cost allocation tags (aws-02)
- ↔ **GCP** Billing budgets, labels, BigQuery export (gcp-02)

COST CHECK

- $ Budgets and Cost Management are free to use
- $ Biggest surprises: running VMs left on, premium SKUs picked by default, data egress
- $ Dev / Test subscriptions (with Visual Studio) get discounted rates

COST TOOLS

TOOL	DOES
Cost analysis	Explore and group spend
Budgets	Alert at thresholds
Anomaly alerts	Flag unusual daily spend
Exports	Daily CSV to storage
Advisor	Rightsize, reserve, delete idle
Pricing calculator	Estimate before building

OFFERS

OFFER	USE
Free account	Credit for 30 days + free services
Pay-as-you-go	Default
Dev/Test	Discounted non-prod
Enterprise / MCA	Company agreements
Reservations / savings plan	Commit for discounts

DO & DON'T

- ✓ **Do** Create a budget per subscription before deploying anything
- ✓ **Do** Require `owner` and `costcenter` tags with Azure Policy
- ✗ **Don't** Rely on the invoice to discover a runaway resource
- ✗ **Don't** Share one subscription for dev and prod

LOCAL TESTING

- ⌂ Estimate with the Azure Pricing Calculator before deploying
- ⌂ Infracost reads Terraform azurerm plans and prices them
- ⌂ Use a Visual Studio subscription credit for experiments

PRODUCTION CHECKLIST

- ☐ Budgets and anomaly alerts routed to the team
- ☐ Tag policy enforced at the management group
- ☐ Monthly Advisor cost review with owners

Azure CLI

SCRIPTING AZURE

The Azure CLI (`az`) is a cross-platform command line for every Azure service. Commands follow `az <group> <subgroup> <verb>`, output JSON by default, and accept `--query` (JMESPath) to pick fields and `-o table|tsv|yaml` to format. Defaults save typing; extensions add preview and newer services.

KEY POINTS

- Install: `brew install azure-cli`, winget, or the Linux script
- `az upgrade` updates the CLI; `az extension add -n <name>` adds features
- Use `-o tsv` to capture values in shell variables
- `az find "create vm"` and `-h` show examples

SECURITY CHECK

- ⚠ Tokens are cached under `~/.azure`; protect that folder like a password
- ⚠ Never pass `--password` on the command line in scripts: use managed identity or OIDC
- ⚠ Azure now requires MFA for CLI sign-ins by users

EVERYDAY AZ

```
az login                                    # browser sign-in
az login --use-device-code                  # SSH / no browser
az account set --subscription "shop-prod"
az account show --query "{name:name, id:id}" -o table

az config set defaults.group=rg-shop-dev defaults.location=eastus2
az config set core.output=table

# JMESPath: names of running VMs
az vm list -d --query "[?powerState=='VM running'].name" -o tsv

# Capture an ID for the next command
SUB=$(az account show --query id -o tsv)
STG=$(az storage account show -n shopdevdata --query id -o tsv)

az extension add --name containerapp --upgrade
az extension list -o table
az group delete -n rg-scratch --yes --no-wait   # async cleanup

az interactive                              # autocomplete shell
```

CODE EXPLAINED

- → `az account set` picks the subscription every later command uses
- → Defaults in `az config` apply until you change them
- → `-d` adds runtime details such as power state to `vm list`
- → `--no-wait` returns immediately while Azure deletes in the background

AWS / GCP EQUIVALENT

- ↔ **AWS** AWS CLI v2, `--query`, profiles (aws-03)
- ↔ **GCP** gcloud, `--format` / `--filter`, configurations (gcp-03)
- ↔ Both az and aws use JMESPath for `--query`

COST CHECK

- $ The CLI is free; Cloud Shell storage (a file share) costs a little
- $ A typo in `--sku` can create a premium tier: check before running
- $ `--no-wait` deletes still bill until they finish

OUTPUT FORMATS

FLAG	USE
`-o json`	Default, full detail
`-o table`	Humans
`-o tsv`	Shell variables
`-o yaml`	Readable config
`-o none`	Quiet scripts

HANDY FLAGS

FLAG	DOES
`--query`	JMESPath filter
`--subscription`	One-off subscription
`--no-wait`	Don't block
`--debug`	Show HTTP calls
`--only-show-errors`	Quieter output

DO & DON'T

- ✓ **Do** Set defaults for group and location in interactive sessions
- ✓ **Do** Pass `--subscription` explicitly in scripts
- ✗ **Don't** Parse `-o table` output in scripts (use tsv or json)
- ✗ **Don't** Store service principal secrets in shell history

LOCAL TESTING

- ⌂ Run the CLI in Docker: `docker run -it mcr.microsoft.com/azure-cli`
- ⌂ Cloud Shell in the browser needs no install
- ⌂ Use `--debug` to see the REST calls a command makes

PRODUCTION CHECKLIST

- ☐ Automation signs in with managed identity or OIDC, not passwords
- ☐ Scripts pin the subscription and fail on errors (`set -euo pipefail`)
- ☐ CLI version pinned in CI images

SDKs & Credentials

DefaultAzureCredential · credential chain order · azure-identity for Python & JavaScript · environment variables · managed identity over secrets · Connect-AzAccount pointer

AUTHENTICATING CODE

Azure SDKs authenticate with token credentials from the `azure-identity` library. **DefaultAzureCredential** tries a chain: environment variables, workload identity, managed identity, then developer tools such as the Azure CLI. The same code uses your `az login` on a laptop and the app's managed identity in Azure, with no secrets in code.

KEY POINTS

- Python: `pip install azure-identity azure-storage-blob`
- JavaScript: `npm i @azure/identity @azure/storage-blob`
- Assign data roles (e.g. Storage Blob Data Reader) to the identity
- In production, a specific credential (ManagedIdentityCredential) avoids slow fallbacks

SECURITY CHECK

- ⚠ Prefer managed identity; client secrets expire and leak
- ⚠ Disable shared keys on storage once identity access works
- ⚠ Grant data-plane roles, not Owner, to app identities

PYTHON AND JAVASCRIPT

```python
from azure.identity import DefaultAzureCredential, ManagedIdentityCredential
from azure.storage.blob import BlobServiceClient

cred = DefaultAzureCredential()            # laptop: az login; Azure: MI
svc = BlobServiceClient("https://shopdata.blob.core.windows.net", cred)
for c in svc.list_containers():
    print(c.name)

prod_cred = ManagedIdentityCredential(client_id="<user-assigned-mi-id>")
```

JAVASCRIPT

```javascript
import { DefaultAzureCredential } from "@azure/identity";
import { BlobServiceClient } from "@azure/storage-blob";
const svc = new BlobServiceClient(
  "https://shopdata.blob.core.windows.net", new DefaultAzureCredential());
for await (const c of svc.listContainers()) console.log(c.name);
```

ENVIRONMENT FOR A SERVICE PRINCIPAL (CI ONLY)

```
export AZURE_TENANT_ID=... AZURE_CLIENT_ID=... AZURE_CLIENT_SECRET=...
```

CODE EXPLAINED

- → No key or connection string appears in the code
- → `client_id` selects a user-assigned identity when several exist
- → The JS SDK uses the same credential pattern
- → Environment variables are the first link in the chain

AWS / GCP EQUIVALENT

- ↔ **AWS** Credential provider chain, boto3 / JS SDK v3 (aws-04)
- ↔ **GCP** Application Default Credentials (gcp-04)

COST CHECK

- $ SDKs are free; you pay for the calls to billable services
- $ Token requests to Entra ID are free
- $ Key Vault calls for secrets are billed per 10,000 operations

CREDENTIAL CHAIN (DEFAULTAZURECREDENTIAL)

ORDER	CREDENTIAL	WHERE
1	Environment	AZURE_* variables
2	Workload identity	AKS pods, federated
3	Managed identity	Azure compute
4+	Developer tools	VS Code, Azure CLI, PowerShell, azd

PICK A CREDENTIAL

WHERE	USE
Laptop	AzureCliCredential (az login)
App Service / Functions / VMs	ManagedIdentityCredential
AKS	WorkloadIdentityCredential
GitHub Actions	azure/login with OIDC

DO & DON'T

- ✓ **Do** Use DefaultAzureCredential in dev and a specific credential in prod
- ✓ **Do** Reuse one client per service across requests
- ✗ **Don't** Put connection strings with account keys in config files
- ✗ **Don't** Give the app identity Contributor when it only reads blobs

LOCAL TESTING

- ⌂ `az login` makes DefaultAzureCredential work locally
- ⌂ Azurite accepts the well-known dev account key or connection string `UseDevelopmentStorage=true`
- ⌂ Set `AZURE_LOG_LEVEL` / SDK logging to debug auth fallbacks

PRODUCTION CHECKLIST

- ☐ Managed identity with least-privilege data roles
- ☐ Shared key access disabled where supported
- ☐ No client secrets; if unavoidable, stored in Key Vault and rotated

Microsoft Entra ID

tenants · users & groups · app registrations vs service principals (enterprise apps) · conditional access · MFA · `az ad` commands

AZURE'S IDENTITY DIRECTORY

Microsoft Entra ID (formerly Azure Active Directory) holds users, groups and application identities for a **tenant**. An **app registration** defines an application once; a **service principal** (shown as an enterprise application) is its instance in a tenant and is what you grant roles to. **Conditional Access** decides when sign-ins need MFA, a compliant device or are blocked.

KEY POINTS

- Entra ID controls sign-in; Azure RBAC (sheet 06) controls what you can do in Azure
- Group-based access scales better than per-user grants
- Managed identities are service principals Azure manages for you
- Conditional Access needs Entra ID P1 or higher

USERS, GROUPS AND APP IDENTITIES

```
# Users and groups
az ad user create --display-name "Ada Lovelace" \
    --user-principal-name ada@contoso.onmicrosoft.com \
    --password '<temp>' --force-change-password-next-sign-in true
az ad group create --display-name sg-shop-devs --mail-nickname sg-shop-devs
az ad group member add --group sg-shop-devs \
    --member-id $(az ad user show --id ada@contoso.onmicrosoft.com \
                  --query id -o tsv)

# App registration + service principal
APP=$(az ad app create --display-name shop-api --query appId -o tsv)
az ad sp create --id $APP
az ad sp list --display-name shop-api --query "[].{app:appId, id:id}" -o table

# Who am I and what tenant
az ad signed-in-user show --query userPrincipalName
az account show --query tenantId -o tsv
```

CODE EXPLAINED

- → Users get a temporary password and must change it at first sign-in
- → Groups are the unit you grant roles to
- → An app registration plus `sp create` gives an identity you can grant roles
- → Object IDs (not names) are what role assignments use

AWS / GCP EQUIVALENT

- ↔ **AWS** IAM Identity Center / IAM users and roles (aws-05)
- ↔ **GCP** Cloud Identity / Google Workspace accounts, groups (gcp-05)
- ↔ Entra ID also handles SSO for Microsoft 365 and SaaS apps

COST CHECK

- $ Entra ID Free comes with every subscription
- $ Conditional Access and PIM need P1 / P2 licenses
- $ Guest (B2B) users are free up to generous monthly active user limits

IDENTITY TYPES

TYPE	USE
User	A person
Group	Assign access to many
App registration	App definition (one tenant or multi)
Service principal	App identity in a tenant
Managed identity	Azure-managed workload identity
Guest (B2B)	External collaborator

LICENSES

TIER	ADDS
Free	Users, groups, basic SSO
P1	Conditional Access, dynamic groups
P2	PIM, identity protection
Governance	Access reviews at scale

DO & DON'T

- ✓ **Do** Require MFA for every user (security defaults or Conditional Access)
- ✓ **Do** Keep two break-glass Global Administrator accounts excluded and monitored
- ✗ **Don't** Use Global Administrator for daily work
- ✗ **Don't** Create client secrets for apps that can use managed identity

LOCAL TESTING

- ⌂ Use a free developer tenant (Microsoft 365 Developer Program, if eligible) for experiments
- ⌂ `az ad` commands work in Cloud Shell with your user
- ⌂ jwt.ms decodes Entra ID tokens to inspect claims

PRODUCTION CHECKLIST

- ☐ MFA enforced and legacy authentication blocked
- ☐ Privileged Identity Management for admin roles
- ☐ Sign-in and audit logs exported to Log Analytics

Azure RBAC

role definitions · role assignments · scope (management group → subscription → resource group → resource) · Owner / Contributor / Reader · custom roles · `az role assignment create`

WHO CAN DO WHAT IN AZURE

Azure role-based access control grants permissions by assigning a **role** to a **principal** at a **scope**. A role definition lists allowed actions (management plane) and data actions (reading blobs, secrets, messages). Assignments inherit downward: a role at a subscription applies to every resource group and resource inside it.

KEY POINTS

- Owner = full control including access; Contributor = full control minus access; Reader = view
- Data access needs data roles (e.g. Storage Blob Data Contributor), not Contributor
- Role changes can take a few minutes to apply
- Prefer built-in roles; there are hundreds

LIMITS & QUOTAS

- ≤ 4,000 role assignments per subscription
- ≤ 5,000 custom roles per tenant
- ≤ Assignments at management groups: 500 per group

ASSIGN, LIST, CUSTOMIZE

```
# Grant a group Contributor on one resource group
az role assignment create --assignee-object-id <group-object-id> \
    --assignee-principal-type Group --role Contributor \
    --scope /subscriptions/<sub-id>/resourceGroups/rg-shop-dev

# Give an app identity read access to blobs in one storage account
az role assignment create --assignee <mi-principal-id> \
    --role "Storage Blob Data Reader" --scope $STG

# What can this principal do, everywhere?
az role assignment list --assignee <object-id> --all -o table
az role definition list --name "Key Vault Secrets User" \
    --query "[].permissions[].dataActions"
```

CODE EXPLAINED

- → Scope is a resource ID; narrower scope means less access
- → `--assignee-principal-type` avoids a directory lookup and race conditions
- → `--all` lists assignments at every scope
- → Create the custom role with `az role definition create --role-definition @vm-operator.json`

CUSTOM ROLE (JSON)

```
{ "Name": "VM Operator",
  "Actions": ["Microsoft.Compute/virtualMachines/start/action",
              "Microsoft.Compute/virtualMachines/restart/action",
              "Microsoft.Compute/virtualMachines/read"],
  "AssignableScopes": ["/subscriptions/<sub-id>"] }
```

AWS / GCP EQUIVALENT

- ↔ **AWS** IAM policies attached to roles / users (aws-06)
- ↔ **GCP** IAM roles bound in allow policies (gcp-05, gcp-06)
- ↔ AWS policies are documents you attach; Azure assigns a role at a scope

COST CHECK

- $ RBAC is free
- $ Over-broad roles cost money indirectly: anyone with Contributor can create premium resources
- $ Use Azure Policy to cap allowed SKUs if needed

KEY BUILT-IN ROLES

ROLE	CAN
`Owner`	Everything + grant access
`Contributor`	Everything except grant access
`Reader`	View only
`User Access Administrator`	Grant access only
`Storage Blob Data Contributor`	Read / write blobs
`Key Vault Secrets User`	Read secrets
`AcrPull`	Pull images

SCOPE ORDER

SCOPE	EXAMPLE
`Management group`	/providers/Microsoft.Management/managementGroups/mg-prod
`Subscription`	/subscriptions/<id>
`Resource group`	…/resourceGroups/rg-shop
`Resource`	…/providers/Microsoft.KeyVault/vaults/kv-shop

DO & DON'T

- ✓ **Do** Assign roles to groups and managed identities
- ✓ **Do** Use PIM for time-limited Owner access
- ✗ **Don't** Grant Owner at subscription scope to developers
- ✗ **Don't** Use Contributor where a data role is what the app needs

LOCAL TESTING

- ⌂ Test in a sandbox resource group with your own Reader-only test user
- ⌂ `az role assignment list --assignee` audits effective grants
- ⌂ Azure Portal "Check access" shows effective permissions

PRODUCTION CHECKLIST

- □ No standing Owner assignments except break-glass
- □ Assignments managed in IaC and reviewed
- □ Alerts on role assignment changes (Activity Log)

Sheet 07

Managed Identities & Federation

system- vs user-assigned identities · granting roles to an identity · federated credentials (GitHub Actions OIDC) · workload identity for AKS · no secrets in code

IDENTITIES AZURE MANAGES FOR YOU

A **managed identity** is a service principal Azure creates and rotates automatically. **System-assigned** identities live and die with one resource; **user-assigned** identities are standalone and can be shared and pre-authorized. **Federated credentials** let outside workloads (GitHub Actions, Kubernetes pods, other clouds) exchange their own OIDC tokens for Azure tokens: no secrets anywhere.

KEY POINTS

- Enable on App Service, Functions, Container Apps, VMs, AKS and more
- Grant roles to the identity's principal ID
- User-assigned identities avoid role-assignment delays on new resources
- Federated credential subjects pin exactly which repo / branch / environment may log in

SECURITY CHECK

- ⚠ Scope federated subjects tightly (`repo:org/app:environment:production`)
- ⚠ Avoid wildcard-like broad subjects and shared identities across apps
- ⚠ Remove client secrets from app registrations once federation works

CREATE, ATTACH, FEDERATE

```
# User-assigned identity attached to a web app
az identity create -g rg-shop -n id-shop-api
MI=$(az identity show -g rg-shop -n id-shop-api --query principalId -o tsv)
az webapp identity assign -g rg-shop -n app-shop-api \
    --identities $(az identity show -g rg-shop -n id-shop-api --query id -o tsv)
az role assignment create --assignee-object-id $MI \
    --assignee-principal-type ServicePrincipal \
    --role "Key Vault Secrets User" --scope $KV

# System-assigned on a VM
az vm identity assign -g rg-shop -n vm-worker

# Trust GitHub Actions for production deploys (no secret)
az identity federated-credential create -g rg-shop \
    --identity-name id-shop-deploy --name gh-prod \
    --issuer https://token.actions.githubusercontent.com \
    --subject repo:contoso/shop:environment:production \
    --audiences api://AzureADTokenExchange
```

CODE EXPLAINED

- → The identity gets roles like any principal
- → `--assignee-principal-type ServicePrincipal` avoids lookup delays for new identities
- → The federated subject must match the GitHub token claim exactly
- → GitHub then uses `azure/login` with the identity's client ID (sheet 39)

AWS / GCP EQUIVALENT

- ↔ **AWS** IAM roles, instance profiles, OIDC providers (aws-07)
- ↔ **GCP** Service accounts, Workload Identity Federation (gcp-07)

COST CHECK

- $ Managed identities and federated credentials are free
- $ They remove the cost of secret rotation work and incidents
- $ Up to 20 federated credentials per identity / app

SYSTEM VS USER-ASSIGNED

ITEM	SYSTEM	USER
Lifecycle	Tied to resource	Independent
Share across resources	No	Yes
Pre-assign roles	No	Yes
Best for	One-off resources	Fleets, IaC, scale sets

FEDERATION SUBJECTS (GITHUB)

TRIGGER	SUBJECT
Branch	repo:org/app:ref:refs/heads/main
Environment	repo:org/app:environment:production
Pull request	repo:org/app:pull_request
Tag	repo:org/app:ref:refs/tags/v1.0

DO & DON'T

- ✓ **Do** Use user-assigned identities in IaC so roles exist before the app starts
- ✓ **Do** One identity per app or workload
- ✗ **Don't** Store a service principal secret in GitHub secrets
- ✗ **Don't** Reuse the deploy identity as the runtime identity

LOCAL TESTING

- ⌂ Locally, DefaultAzureCredential falls back to your `az login`
- ⌂ Grant your own user the same data role for local testing
- ⌂ The managed identity endpoint only exists inside Azure; do not try to mock it

PRODUCTION CHECKLIST

- ☐ All apps use managed identities; zero client secrets
- ☐ Federated credentials limited to protected environments
- ☐ Identity role assignments reviewed quarterly

Key Vault

secrets, keys & certificates · RBAC vs access policies · soft delete & purge protection · `az keyvault secret set` / `show` · Key Vault references in App Service · Managed HSM pointer

SECRETS, KEYS AND CERTIFICATES

Azure Key Vault stores secrets (passwords, connection strings), cryptographic keys and TLS certificates. Access is controlled by **Azure RBAC** (the default for new vaults) or legacy access policies. **Soft delete** keeps deleted items recoverable, and **purge protection** stops anyone from permanently deleting them early. App Service and Functions can read secrets through **Key Vault references** without code.

KEY POINTS

- Vault names are global: `https://kv-shop-prod.vault.azure.net`
- Secrets are versioned; reference a version or always the latest
- Standard tier uses software keys; Premium adds HSM-backed keys
- Managed HSM is a dedicated, single-tenant HSM pool

VAULT, SECRET, REFERENCE

```
az keyvault create -n kv-shop-prod -g rg-shop -l eastus2 \
    --enable-rbac-authorization true --enable-purge-protection true \
    --public-network-access Disabled
KV=$(az keyvault show -n kv-shop-prod --query id -o tsv)

ME=$(az ad signed-in-user show --query id -o tsv)
az role assignment create --assignee $ME \
    --role "Key Vault Secrets Officer" --scope $KV
az keyvault secret set --vault-name kv-shop-prod -n db-password \
    --value '<from-a-file-or-pipeline>'
az keyvault secret show --vault-name kv-shop-prod -n db-password \
    --query id -o tsv                       # versioned URI

# App setting that resolves the secret at runtime
az webapp config appsettings set -g rg-shop -n app-shop-api --settings \
  "DB_PASS=@Microsoft.KeyVault(VaultName=kv-shop-prod;SecretName=db-password)"

az keyvault secret recover --vault-name kv-shop-prod -n db-password
```

CODE EXPLAINED

- → RBAC authorization and purge protection are set at creation
- → Secrets Officer can write; the app only needs Secrets User
- → The Key Vault reference syntax pulls the latest secret version
- → `recover` undoes a soft delete

AWS / GCP EQUIVALENT

- ↔ **AWS** Secrets Manager, KMS (aws-08)
- ↔ **GCP** Secret Manager, Cloud KMS (gcp-08)
- ↔ Key Vault combines secrets, keys and certificates in one service

READ IN PYTHON

```
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient
c = SecretClient("https://kv-shop-prod.vault.azure.net",
                 DefaultAzureCredential())
print(c.get_secret("db-password").value)
```

COST CHECK

- $ Billed per 10,000 operations; HSM keys and Managed HSM cost more
- $ Certificate renewals through partner CAs carry their own fees
- $ Caching secrets in the app cuts both cost and throttling

SECURITY CHECK

- ⚠ Turn on purge protection for production vaults
- ⚠ Disable public network access; use a private endpoint
- ⚠ Use RBAC roles (Key Vault Secrets User) instead of access policies

LIMITS & QUOTAS

- ≤ Secret value up to 25 KB
- ≤ Throttled per vault per region (thousands of GETs per 10 s); cache secrets
- ≤ Soft delete retention 7–90 days (default 90)

ROLES

ROLE	GRANTS
Key Vault Administrator	All data actions
Key Vault Secrets Officer	Manage secrets
Key Vault Secrets User	Read secret values
Key Vault Crypto User	Use keys
Key Vault Certificates Officer	Manage certificates
Key Vault Reader	Metadata only

DO & DON'T

- ✓ **Do** One vault per app per environment
- ✓ **Do** Read secrets with managed identity and cache them briefly
- ✗ **Don't** Put secret values in Bicep parameters files or pipeline logs
- ✗ **Don't** Use access policies on new vaults

LOCAL TESTING

- ⌂ Use a dev vault in a sandbox subscription; there is no official local emulator
- ⌂ Tests can swap in an in-memory secret provider behind an interface
- ⌂ `az keyvault secret list` checks names without revealing values

PRODUCTION CHECKLIST

- ☐ Purge protection and soft delete on
- ☐ Private endpoint with public access disabled
- ☐ Diagnostic logs to Log Analytics; alerts on access failures

Security & Governance

GUARDRAILS AND THREAT DETECTION

Azure Policy evaluates resources against rules and can deny, audit or fix them, with **initiatives** grouping many policies. **Microsoft Defender for Cloud** scores your security posture and adds threat protection plans per service. The **Activity Log** records every management action, **resource locks** prevent accidental deletes, and **Microsoft Sentinel** is the SIEM for correlating it all.

KEY POINTS

- Policy effects: Deny, Audit, Modify, DeployIfNotExists, Append
- Assign policies at management groups for broad coverage
- Activity Log keeps 90 days; export it for longer retention
- Defender CSPM foundational checks are free; plans per resource type are paid

POLICY, LOCKS, DEFENDER

```
# Require an "owner" tag on resource groups (built-in policy)
DEF=$(az policy definition list \
  --query "[?displayName=='Require a tag on resource groups'].name" -o tsv)
az policy assignment create -n require-owner-tag --policy $DEF \
    --scope /subscriptions/<sub-id> --params '{"tagName":{"value":"owner"}}'
az policy state summarize --subscription <sub-id>

# Prevent deletes on production
az lock create -n no-delete -g rg-shop-prod --lock-type CanNotDelete

# Defender plans and posture
az security pricing create -n VirtualMachines --tier Standard
az security pricing list -o table
az security secure-scores list -o table

# Export Activity Log to Log Analytics
az monitor diagnostic-settings subscription create -n activity-to-law \
    --location eastus2 --workspace <law-id> \
    --logs '[{"category":"Administrative","enabled":true}]'
```

CODE EXPLAINED

- → Built-in policies are found by display name and assigned with parameters
- → `policy state summarize` counts compliant and non-compliant resources
- → A CanNotDelete lock blocks deletes even for Owners until removed
- → Diagnostic settings send Activity Log to a workspace for longer retention

AWS / GCP EQUIVALENT

- ↔ **AWS** CloudTrail, GuardDuty, Security Hub, Config, SCPs (aws-09)
- ↔ **GCP** Audit Logs, Security Command Center, Organization Policy (gcp-09)

COST CHECK

- $ Azure Policy is free for Azure resources
- $ Defender plans bill per resource per hour (servers, storage, SQL…)
- $ Sentinel bills on ingested GB: filter what you send

SERVICES

SERVICE	ROLE
Azure Policy	Prevent and audit configuration
Defender for Cloud	Posture score, threat protection
Activity Log	Who changed what
Resource locks	Block delete / change
Sentinel	SIEM + SOAR
DDoS Protection	Network-level DDoS mitigation
WAF	App Gateway / Front Door rules

POLICY EFFECTS

EFFECT	DOES
Deny	Block the request
Audit	Report only
Modify	Add / fix tags or properties
DeployIfNotExists	Deploy a missing setting
Disabled	Turn off

DO & DON'T

- ✓ **Do** Start policies in Audit, then switch to Deny
- ✓ **Do** Lock production resource groups against deletion
- ✗ **Don't** Assign hundreds of policies to one subscription by hand
- ✗ **Don't** Ignore Defender recommendations because the score looks fine

LOCAL TESTING

- ⌂ Test policy rules against sample JSON with the Azure Policy VS Code extension
- ⌂ PSRule for Azure and Checkov check Bicep / Terraform before deploy
- ⌂ Use a sandbox subscription to verify a Deny policy blocks what you expect

PRODUCTION CHECKLIST

- ☐ Baseline initiative (e.g. Microsoft cloud security benchmark) assigned
- ☐ Activity Log exported and retained per compliance
- ☐ Defender plans on for production workloads

Virtual Networks

address spaces · subnets · system routes & route tables (UDRs) · `az network vnet create` · default outbound access retirement · subnet delegation

PRIVATE NETWORKING ON AZURE

A **virtual network (VNet)** is a regional private address space split into **subnets** that span the region's zones. Azure adds **system routes** automatically; **route tables** (user-defined routes) override them, for example to send traffic through a firewall. Some PaaS services need a **delegated** subnet. New VNets now have **private subnets by default**: VMs need an explicit outbound path.

KEY POINTS

- Default outbound access for new VNets retired on March 31, 2026
- Outbound options: NAT Gateway, load balancer outbound rules, public IP, firewall
- Azure reserves 5 addresses in every subnet
- Some services need dedicated subnets (Application Gateway, Bastion, Firewall)

SECURITY CHECK

- ⚠ Do not attach public IPs to VMs; use Bastion and a load balancer
- ⚠ Put an NSG on every subnet (sheet 12)
- ⚠ Use private endpoints for PaaS services in the VNet (sheet 11)

LIMITS & QUOTAS

- ≤ 1,000 VNets per subscription per region
- ≤ 3,000 subnets per VNet
- ≤ 500 peerings per VNet

VNET, SUBNETS, ROUTES

```
az network vnet create -g rg-net -n vnet-shop-prod -l eastus2 \
    --address-prefixes 10.20.0.0/16 \
    --subnet-name snet-app --subnet-prefixes 10.20.1.0/24
az network vnet subnet create -g rg-net --vnet-name vnet-shop-prod \
    -n snet-data --address-prefixes 10.20.2.0/24
az network vnet subnet create -g rg-net --vnet-name vnet-shop-prod \
    -n snet-aca --address-prefixes 10.20.16.0/23 \
    --delegations Microsoft.App/environments

# Send all outbound traffic through a firewall appliance
az network route-table create -g rg-net -n rt-to-fw
az network route-table route create -g rg-net --route-table-name rt-to-fw \
    -n default --address-prefix 0.0.0.0/0 \
    --next-hop-type VirtualAppliance --next-hop-ip-address 10.20.255.4
az network vnet subnet update -g rg-net --vnet-name vnet-shop-prod \
    -n snet-app --route-table rt-to-fw

az network vnet subnet list -g rg-net --vnet-name vnet-shop-prod -o table
```

CODE EXPLAINED

- → The VNet and its first subnet are created together
- → Container Apps environments need a delegated subnet
- → A 0.0.0.0/0 route to a virtual appliance forces traffic through the firewall
- → Route tables attach per subnet

AWS / GCP EQUIVALENT

- ↔ **AWS** VPC, subnets per AZ, route tables (aws-10)
- ↔ **GCP** Global VPC, regional subnets, routes (gcp-10)
- ↔ Azure subnets span zones like Google Cloud's, unlike AWS's one-AZ subnets

COST CHECK

- $ VNets and subnets are free; peering, gateways and NAT are billed
- $ Data between zones in a region is billed per GB
- $ Unused public IPs bill hourly

OUTBOUND OPTIONS

OPTION	USE
NAT Gateway	Default choice for subnets
Load balancer outbound rules	With a public Standard LB
Instance public IP	Single VM, avoid in prod
Azure Firewall	Inspect and filter egress

DEDICATED SUBNETS

SERVICE	SUBNET
Azure Bastion	AzureBastionSubnet (/26+)
Azure Firewall	AzureFirewallSubnet (/26)
VPN / ExpressRoute gateway	GatewaySubnet (/27+)
Application Gateway	Any dedicated subnet
Container Apps	Delegated (/23 or /27 workload profiles)

DO & DON'T

- ✓ **Do** Plan non-overlapping /16s per environment and region
- ✓ **Do** Give each tier its own subnet with its own NSG
- ✗ **Don't** Rely on implicit outbound internet access for VMs
- ✗ **Don't** Make subnets so small that scaling or new services do not fit

LOCAL TESTING

- ⌂ Practice address plans with `ipcalc` before creating anything
- ⌂ Network Watcher "IP flow verify" and "Next hop" explain routing in Azure
- ⌂ Docker networks give a local feel for subnet isolation

PRODUCTION CHECKLIST

- ☐ Address plan documented and non-overlapping with on-premises
- ☐ Explicit outbound path (NAT Gateway or firewall) for every subnet
- ☐ VNet flow logs enabled

NAT, Private Endpoints & Connectivity

NAT Gateway · service endpoints vs Private Endpoints (Private Link) · Private DNS zones · VNet peering · VPN Gateway · ExpressRoute · Virtual WAN

GETTING IN, OUT AND ACROSS

NAT Gateway gives a subnet a fixed outbound public IP. A **Private Endpoint** places a PaaS service (storage, SQL, Key Vault) on a private IP in your VNet, with a **Private DNS zone** so its normal name resolves privately. **Peering** links VNets; **VPN Gateway** and **ExpressRoute** link on-premises; **Virtual WAN** manages hub-and-spoke at scale.

KEY POINTS

- Private endpoints beat service endpoints: traffic stays on a private IP
- Each PaaS type has a private DNS zone, e.g. `privatelink.blob.core.windows.net`
- Peering is not transitive: spoke A cannot reach spoke B through the hub without routing
- ExpressRoute is a private circuit through a connectivity provider

NAT AND A PRIVATE ENDPOINT FOR BLOB STORAGE

```
az network public-ip create -g rg-net -n pip-nat --sku Standard
az network nat gateway create -g rg-net -n ng-shop --public-ip-addresses pip-nat
az network vnet subnet update -g rg-net --vnet-name vnet-shop-prod \
    -n snet-app --nat-gateway ng-shop

az network private-endpoint create -g rg-net -n pe-shopdata-blob \
    --vnet-name vnet-shop-prod --subnet snet-data \
    --private-connection-resource-id $STG --group-id blob \
    --connection-name shopdata-blob

az network private-dns zone create -g rg-net \
    -n privatelink.blob.core.windows.net
az network private-dns link vnet create -g rg-net -n link-shop \
    --zone-name privatelink.blob.core.windows.net \
    --virtual-network vnet-shop-prod --registration-enabled false
az network private-endpoint dns-zone-group create -g rg-net \
    --endpoint-name pe-shopdata-blob -n default \
    --private-dns-zone privatelink.blob.core.windows.net --zone-name blob

# Peer two VNets (run in both directions)
az network vnet peering create -g rg-net -n hub-to-spoke \
    --vnet-name vnet-hub --remote-vnet vnet-shop-prod --allow-vnet-access
```

CODE EXPLAINED

- → The NAT gateway gives every VM in `snet-app` the same outbound IP
- → `--group-id blob` picks the storage sub-resource to expose
- → The DNS zone group keeps the private A record in sync
- → Peering needs a link from each side

AWS / GCP EQUIVALENT

- ↔ **AWS** NAT Gateway, PrivateLink / VPC endpoints, peering, Transit Gateway (aws-11)
- ↔ **GCP** Cloud NAT, Private Service Connect, peering, NCC (gcp-11)

COST CHECK

- $ NAT Gateway: hourly + per GB processed
- $ Private endpoints: hourly + per GB in and out
- $ Peering bills per GB in each direction; global peering costs more

PRIVATE ACCESS OPTIONS

OPTION	PRIVATE IP	NOTES
Private Endpoint	Yes	Recommended; needs private DNS
Service endpoint	No	Free; source stays public IP
Public + firewall rules	No	Allow-list IPs

CONNECTIVITY

NEED	USE
VNet ↔ VNet	Peering
Office ↔ Azure (internet)	VPN Gateway
Datacenter ↔ Azure (private)	ExpressRoute
Many sites and VNets	Virtual WAN
Hub routing / inspection	Azure Firewall in hub

DO & DON'T

- ✓ **Do** Centralize private DNS zones in a hub subscription
- ✓ **Do** Disable public network access once private endpoints work
- ✗ **Don't** Forget DNS: a private endpoint without the zone still resolves publicly
- ✗ **Don't** Assume peering chains route automatically

LOCAL TESTING

- ⌂ `nslookup shopdata.blob.core.windows.net` from a VM shows whether DNS returns the private IP
- ⌂ Network Watcher connection troubleshoot tests paths
- ⌂ Practice hub-and-spoke in a sandbox with tiny VMs

PRODUCTION CHECKLIST

- ☐ PaaS services reachable only through private endpoints
- ☐ Private DNS zones linked to every VNet that needs them
- ☐ Fixed outbound IPs (NAT Gateway) shared with partners who allow-list

NSGs & Azure Firewall

FILTERING TRAFFIC

A **network security group (NSG)** holds stateful allow / deny rules, attached to a subnet or a NIC. Rules run in **priority** order (100 first, 4096 last) and can use **service tags** (Internet, AzureLoadBalancer, Storage.EastUS2) and **application security groups** instead of IPs. **Azure Firewall** is a managed central firewall for hub networks with FQDN filtering and threat intelligence.

KEY POINTS

- Default rules (priority 65000+) allow VNet-to-VNet and load balancer probes, deny other inbound
- NSG on subnet and NIC: both must allow the traffic
- ASGs group VMs by role, e.g. `asg-web`, `asg-db`
- VNet flow logs replace NSG flow logs

SECURITY CHECK

- ⚠ Never allow 22 / 3389 from `Internet`; use Azure Bastion
- ⚠ Review rules with "Any" source or destination
- ⚠ Deny outbound to the internet from data subnets

LIMITS & QUOTAS

- ≤ 1,000 rules per NSG
- ≤ 5,000 NSGs per subscription
- ≤ Priorities 100–4096 for custom rules

NSG WITH SERVICE TAGS AND ASGS

```
az network nsg create -g rg-net -n nsg-app
az network asg create -g rg-net -n asg-web
az network asg create -g rg-net -n asg-db

az network nsg rule create -g rg-net --nsg-name nsg-app -n allow-https \
    --priority 100 --direction Inbound --access Allow --protocol Tcp \
    --source-address-prefixes Internet --destination-port-ranges 443 \
    --destination-asgs asg-web
az network nsg rule create -g rg-net --nsg-name nsg-app -n web-to-db \
    --priority 110 --direction Inbound --access Allow --protocol Tcp \
    --source-asgs asg-web --destination-asgs asg-db \
    --destination-port-ranges 5432
az network nsg rule create -g rg-net --nsg-name nsg-app -n deny-inet-out \
    --priority 4000 --direction Outbound --access Deny \
    --destination-address-prefixes Internet --destination-port-ranges '*'

az network vnet subnet update -g rg-net --vnet-name vnet-shop-prod \
    -n snet-app --network-security-group nsg-app
az network nsg rule list -g rg-net --nsg-name nsg-app --include-default -o table
```

CODE EXPLAINED

- → The `Internet` service tag stands for all public addresses
- → ASGs let rules say "web may reach db" without IPs
- → A low-priority outbound deny blocks internet egress
- → NSGs attach to subnets (preferred) or individual NICs

AWS / GCP EQUIVALENT

- ↔ **AWS** Security groups (stateful), NACLs (stateless) (aws-12)
- ↔ **GCP** VPC firewall rules and firewall policies (gcp-12)
- ↔ NSGs can sit on subnets, which AWS security groups cannot

COST CHECK

- $ NSGs and ASGs are free
- $ Azure Firewall bills hourly plus per GB processed: one per hub, not per spoke
- $ Flow logs bill for storage and optional traffic analytics

RULE FIELDS

FIELD	EXAMPLE
Priority	100 (lower first)
Direction	Inbound / Outbound
Source	Internet, 10.0.0.0/8, asg-web
Destination port	443, 5432, *
Protocol	Tcp, Udp, Icmp, *
Access	Allow / Deny

FIREWALL OPTIONS

NEED	USE
Per-subnet filtering	NSG
Central egress + FQDN rules	Azure Firewall
Web app protection	WAF on App Gateway / Front Door
Third-party NVA	Marketplace appliance

DO & DON'T

- ✓ **Do** Leave gaps between priorities (100, 110, 120) for later rules
- ✓ **Do** Use service tags for Azure services instead of IP ranges
- ✗ **Don't** Open broad port ranges to `Internet`
- ✗ **Don't** Stack NIC and subnet NSGs without documenting why

LOCAL TESTING

- ⌂ Network Watcher "IP flow verify" says which rule allows or blocks a packet
- ⌂ "Effective security rules" on a NIC shows the merged rule set
- ⌂ Test from a VM with `nc -zv` / `curl` after each change

PRODUCTION CHECKLIST

- ☐ NSG on every subnet; no SSH / RDP from Internet
- ☐ Central egress through Azure Firewall where required
- ☐ VNet flow logs with traffic analytics enabled

Sheet 13

Azure DNS & Traffic Manager

public & private zones · record sets · alias records · Traffic Manager routing methods · App Service domains

NAMES AND GLOBAL ROUTING

Azure DNS hosts public zones on Microsoft's anycast name servers and **private zones** that resolve only inside linked VNets. **Alias records** point a name at an Azure resource (public IP, Front Door, Traffic Manager) and follow it if its IP changes. **Traffic Manager** is DNS-based global routing: priority failover, weighted, performance or geographic.

KEY POINTS

- Delegate a domain by setting the registrar's NS records to the zone's name servers
- Alias records can sit at the zone apex where CNAMEs cannot
- Private zones handle `privatelink.*` names for private endpoints
- Traffic Manager works at DNS level; clients cache by TTL

ZONES, RECORDS, TRAFFIC MANAGER

```
az network dns zone create -g rg-dns -n contoso-shop.com
az network dns zone show -g rg-dns -n contoso-shop.com --query nameServers

az network dns record-set a add-record -g rg-dns -z contoso-shop.com \
    -n api -a 20.42.10.5 --ttl 300
az network dns record-set cname set-record -g rg-dns -z contoso-shop.com \
    -n www -c shop.azurefd.net
az network dns record-set txt add-record -g rg-dns -z contoso-shop.com \
    -n @ -v "v=spf1 include:spf.protection.outlook.com -all"
# Alias A record at the apex pointing to a public IP resource
az network dns record-set a create -g rg-dns -z contoso-shop.com -n @ \
    --target-resource $(az network public-ip show -g rg-net -n pip-agw \
                        --query id -o tsv)

# Traffic Manager: primary region, failover to secondary
az network traffic-manager profile create -g rg-dns -n tm-shop \
    --routing-method Priority --unique-dns-name tm-contoso-shop \
    --monitor-path /health --protocol HTTPS --port 443
az network traffic-manager endpoint create -g rg-dns --profile-name tm-shop \
    -n eus2 --type azureEndpoints --priority 1 --target-resource-id <app-id>
```

CODE EXPLAINED

- → Copy the zone's name servers to your registrar to delegate
- → TTL 300 keeps changes quick to propagate
- → The alias record follows the public IP resource, not a fixed address
- → Traffic Manager health-checks `/health` and fails over by priority

AWS / GCP EQUIVALENT

- ↔ **AWS** Route 53 hosted zones, alias, routing policies (aws-13)
- ↔ **GCP** Cloud DNS zones and routing policies (gcp-13)
- ↔ Traffic Manager ≈ Route 53 routing policies; Front Door adds L7

COST CHECK

- $ DNS zones bill monthly per zone plus per million queries
- $ Traffic Manager bills per million queries and per health-checked endpoint
- $ Alias queries to Azure resources are free

RECORD TYPES

TYPE	USE
`A / AAAA`	Name → IP
`CNAME`	Name → name (not at apex)
`Alias (A/AAAA/CNAME)`	Name → Azure resource
`MX`	Mail
`TXT`	SPF, verification
`CAA`	Allowed certificate authorities

TRAFFIC MANAGER METHODS

METHOD	ROUTES BY
`Priority`	Failover order
`Weighted`	Percentages
`Performance`	Lowest latency
`Geographic`	User location
`MultiValue`	Several healthy IPs
`Subnet`	Client IP ranges

DO & DON'T

- ✓ **Do** Keep zones in IaC with records reviewed like code
- ✓ **Do** Add CAA records for your certificate authority
- ✗ **Don't** Point a CNAME at the zone apex (use an alias record)
- ✗ **Don't** Set long TTLs on records you may fail over

LOCAL TESTING

- ⌂ `dig NS contoso-shop.com` confirms delegation
- ⌂ `dig @<azure-ns> api.contoso-shop.com` tests a record before delegation
- ⌂ CoreDNS or dnsmasq in Docker for local name experiments

PRODUCTION CHECKLIST

- ☐ Zone records managed in IaC
- ☐ Health-checked failover for critical names
- ☐ DNSSEC signing considered for public zones

Front Door & Certificates

GLOBAL ENTRY POINT

Azure Front Door is Microsoft's global layer-7 entry point: CDN caching, TLS termination with **managed certificates**, routing to **origins** in any region, health probes and failover, and a **WAF**. A profile holds endpoints; **routes** map domains and paths to **origin groups**. Premium adds Private Link origins and managed WAF rule sets with bot protection.

KEY POINTS

- Front Door (classic) retires on March 31, 2027: use Standard or Premium
- Origins can be App Service, Container Apps, Storage, load balancers or any public host
- Rules engine rewrites headers, redirects and changes cache behavior
- Purge clears cached paths after a deploy

SECURITY CHECK

- ⚠ Lock origins to Front Door only (Private Link or the `X-Azure-FDID` header + service tag)
- ⚠ Enable WAF in Prevention mode after tuning
- ⚠ Redirect HTTP to HTTPS on every route

PROFILE, ORIGIN, ROUTE, WAF

```
az afd profile create -g rg-web --profile-name afd-shop \
    --sku Premium_AzureFrontDoor
az afd endpoint create -g rg-web --profile-name afd-shop --endpoint-name shop

az afd origin-group create -g rg-web --profile-name afd-shop \
    --origin-group-name og-app --probe-path /health \
    --probe-protocol Https --probe-request-type GET \
    --probe-interval-in-seconds 30 --sample-size 4 \
    --successful-samples-required 3 \
    --additional-latency-in-milliseconds 50
az afd origin create -g rg-web --profile-name afd-shop \
    --origin-group-name og-app --origin-name eus2 \
    --host-name app-shop-eus2.azurewebsites.net \
    --origin-host-header app-shop-eus2.azurewebsites.net \
    --priority 1 --weight 1000 --enabled-state Enabled

az afd route create -g rg-web --profile-name afd-shop --endpoint-name shop \
    --route-name default --origin-group og-app \
    --supported-protocols Https Http --https-redirect Enabled \
    --forwarding-protocol HttpsOnly --link-to-default-domain Enabled

az afd endpoint purge -g rg-web --profile-name afd-shop \
    --endpoint-name shop --content-paths "/index.html"
```

CODE EXPLAINED

- → Premium is needed for Private Link origins and managed WAF rules
- → The origin group health probe decides which origins get traffic
- → Origin host header must match what the back end expects
- → Routes tie the endpoint to the origin group and enforce HTTPS

AWS / GCP EQUIVALENT

- ↔ **AWS** CloudFront + ACM + AWS WAF; Global Accelerator (aws-14)
- ↔ **GCP** Global external Application LB + Cloud CDN + Cloud Armor (gcp-14)
- ↔ Front Door combines CDN, global L7 load balancing and WAF

COST CHECK

- $ Base fee per profile per month plus requests and data transfer
- $ Premium costs much more than Standard: use it when you need Private Link or managed rules
- $ Caching static content cuts origin compute and egress

STANDARD VS PREMIUM

FEATURE	STANDARD	PREMIUM
CDN + L7 routing	Yes	Yes
Managed certificates	Yes	Yes
Custom WAF rules	Yes	Yes
Managed WAF rule sets / bot	No	Yes
Private Link origins	No	Yes

PIECES

PIECE	HOLDS
Profile	Endpoints, WAF, rules
Endpoint	*.azurefd.net hostname
Origin group	Origins + health probe
Route	Domain/path → origin group
Rule set	Rewrites, redirects, caching

DO & DON'T

- ✓ **Do** Cache static assets with long TTLs and versioned names
- ✓ **Do** Use custom domains with managed certificates
- ✗ **Don't** Leave origins reachable directly from the internet
- ✗ **Don't** Purge `/*` on every release

LOCAL TESTING

- ⌂ Test origins locally first; Front Door has no emulator
- ⌂ `curl -I https://shop.azurefd.net/app.js` shows cache headers (`x-cache`)
- ⌂ Use a Standard profile in a sandbox to try routing rules cheaply

PRODUCTION CHECKLIST

- ☐ Origins locked to Front Door (Private Link or FDID header check)
- ☐ WAF policy in Prevention mode with logging
- ☐ Two origins in different regions with health probes

Virtual Machines

AZURE VIRTUAL MACHINES

A VM is a size (vCPUs, memory), an image, managed disks and a network interface in a subnet. Size names encode the family and features: `Standard_D2s_v5` is general purpose, 2 vCPUs, premium-storage capable, version 5; `Dpsv6` sizes run on Microsoft's Arm-based **Cobalt** CPUs. **cloud-init** configures Linux on first boot, and **Azure Bastion** gives browser or native SSH without public IPs.

KEY POINTS

- Images: `Ubuntu2404`, `Debian12`, `Win2022Datacenter`, or a Compute Gallery image
- Trusted Launch (Secure Boot, vTPM) is the default for supported Gen2 images
- Zones: pass `--zone 1` to pin a VM to a zone
- B-series burstable sizes suit dev and light workloads

SECURITY CHECK

- ⚠ No public IP: use Bastion or Entra ID login through Bastion
- ⚠ Enable encryption at host for temp disk and cache encryption
- ⚠ Patch with Azure Update Manager, not by hand

LIMITS & QUOTAS

- ≤ vCPU quotas per region and per VM family (new subscriptions start low)
- ≤ Check with `az vm list-usage -l <region>`; request increases in Quotas
- ≤ Not every size exists in every region or zone

CREATE A PRIVATE LINUX VM

```
az vm create -g rg-app -n vm-web-1 -l eastus2 --zone 1 \
    --image Ubuntu2404 --size Standard_D2ps_v6 \
    --admin-username azureuser --generate-ssh-keys \
    --vnet-name vnet-shop-prod --subnet snet-app \
    --public-ip-address "" --nsg "" \
    --custom-data cloud-init.yaml --assign-identity

az vm list-sizes -l eastus2 --query "[?contains(name,'pds_v6')].name" -o tsv
az vm image list --publisher Canonical --offer ubuntu-24_04-lts --all -o table

# SSH through Bastion with your Entra ID account
az network bastion ssh -g rg-net -n bas-shop --auth-type AAD \
    --target-resource-id $(az vm show -g rg-app -n vm-web-1 --query id -o tsv)
az vm run-command invoke -g rg-app -n vm-web-1 \
    --command-id RunShellScript --scripts "systemctl status nginx"
```

CODE EXPLAINED

- → `--public-ip-address ""` and `--nsg ""` skip the public IP and NIC NSG (subnet NSG applies)
- → D2ps_v6 is a 2-vCPU Arm (Cobalt) general-purpose size
- → `--assign-identity` adds a system-assigned managed identity
- → `run-command` executes a script without SSH

AWS / GCP EQUIVALENT

- ↔ **AWS** EC2, AMIs, user data, Session Manager (aws-15)
- ↔ **GCP** Compute Engine, images, startup scripts, OS Login / IAP (gcp-15)

CLOUD-INIT.YAML

```
#cloud-config
package_upgrade: true
packages: [nginx]
runcmd:
  - systemctl enable --now nginx
```

COST CHECK

- $ Billed per minute while running, plus disks and IPs always
- $ Arm (Cobalt) and B-series sizes are cheapest for many Linux workloads
- $ Bastion has an hourly cost: share one per hub

SIZE NAME DECODER

PART	MEANING
`D / E / F / B / N`	General / memory / compute / burstable / GPU
`2`	vCPUs
`p`	Arm-based processor
`d`	Local temp disk
`s`	Premium storage capable
`v6`	Generation

ACCESS OPTIONS

OPTION	USE
`Bastion (Entra ID)`	Production admin access
`run-command`	One-off scripts
`Serial console`	Boot problems
`Public IP + SSH`	Never in production

DO & DON'T

- ✓ **Do** Bake golden images in Azure Compute Gallery for faster, consistent boots
- ✓ **Do** Put VMs in scale sets even for one instance
- ✗ **Don't** Open SSH to the internet with a public IP
- ✗ **Don't** Resize production VMs by hand during business hours

LOCAL TESTING

- ⌂ Multipass or Vagrant runs the same cloud-init file locally
- ⌂ `cloud-init schema --config-file cloud-init.yaml` validates the file
- ⌂ Packer builds Azure images from a local template

PRODUCTION CHECKLIST

- ☐ No public IPs; Bastion or private access only
- ☐ Update Manager patch schedules and Azure Monitor agent installed
- ☐ Zones or scale sets for redundancy; backups enabled

VM Operations & Pricing

RUNNING VMS CHEAPLY AND SAFELY

VMs keep their **managed disks** (Premium SSD v2, Premium SSD, Standard SSD, Ultra) across restarts; **snapshots** copy a disk at a point in time. Stopping from inside the OS keeps billing compute: you must **deallocate** to stop paying. Discounts come from **Spot VMs** (evictable), **Reservations** and the **savings plan for compute**, and **Azure Hybrid Benefit** for existing Windows / SQL licenses.

KEY POINTS

- `az vm stop` powers off but still bills compute; `az vm deallocate` stops it
- Deallocated VMs still pay for disks and static public IPs
- Spot VMs can be evicted with 30 seconds notice
- Auto-shutdown schedules stop dev VMs every evening

OPERATE AND SAVE

```
az vm deallocate -g rg-app -n vm-web-1          # stop paying for compute
az vm start -g rg-app -n vm-web-1
az vm resize -g rg-app -n vm-web-1 --size Standard_D4ps_v6
az vm auto-shutdown -g rg-app -n vm-dev-1 --time 1900   # UTC, daily

# Disks and snapshots
az disk create -g rg-app -n disk-data --size-gb 256 --sku PremiumV2_LRS --zone 1
az vm disk attach -g rg-app --vm-name vm-web-1 --name disk-data
az snapshot create -g rg-app -n snap-data-20261112 --source disk-data \
    --incremental true

# Spot VM that deallocates on eviction, pay up to on-demand price
az vm create -g rg-batch -n vm-spot-1 --image Ubuntu2404 \
    --size Standard_D4ps_v6 --priority Spot --max-price -1 \
    --eviction-policy Deallocate --generate-ssh-keys

# Windows license you already own
az vm update -g rg-app -n vm-win-1 --license-type Windows_Server

az disk list --query "[?diskState=='Unattached'].name" -o tsv  # waste
```

CODE EXPLAINED

- → Deallocate releases the host; the VM keeps its disks and config
- → Incremental snapshots store only changed blocks
- → `--max-price -1` means never evict on price, only on capacity
- → Unattached disks are a common forgotten cost

AWS / GCP EQUIVALENT

- ↔ **AWS** EBS, snapshots, Spot, Savings Plans / RIs (aws-16)
- ↔ **GCP** Persistent Disk / Hyperdisk, Spot VMs, CUDs / SUDs (gcp-16)
- ↔ AWS "stop" stops billing compute; Azure needs deallocate

COST CHECK

- $ Reservations: up to ~72% off for 1 or 3 years on a fixed size
- $ Savings plan for compute: flexible across sizes and regions, lower discount
- $ Spot: up to ~90% off, eviction at any time

DISK TYPES

TYPE	USE
Ultra Disk	Highest IOPS, tunable
Premium SSD v2	Tunable IOPS/throughput, good default for prod
Premium SSD	Production, fixed tiers
Standard SSD	Dev, light prod
Standard HDD	Backup, infrequent

POWER STATES

STATE	COMPUTE BILLED	DISK BILLED
Running	Yes	Yes
Stopped (in OS / az vm stop)	Yes	Yes
Deallocated	No	Yes
Deleted	No	Only if disks kept

DO & DON'T

- ✓ **Do** Deallocate (or auto-shutdown) dev VMs outside working hours
- ✓ **Do** Rightsize with Advisor before buying Reservations
- ✗ **Don't** Shut down from inside the OS and assume billing stopped
- ✗ **Don't** Run stateful, uninterruptible workloads on Spot

LOCAL TESTING

- ⌂ Simulate Spot eviction with `az vm simulate-eviction` in a sandbox
- ⌂ Compare prices in the Azure Pricing Calculator
- ⌂ Test snapshot restores into a scratch resource group

PRODUCTION CHECKLIST

- ☐ Backups or snapshot schedules for data disks
- ☐ Reservations / savings plan cover the steady baseline
- ☐ Alerts for unattached disks and unused public IPs

Load Balancing

SPREADING TRAFFIC

Azure has four balancers. **Azure Load Balancer** is layer 4 (TCP / UDP), regional, very fast. **Application Gateway** is a regional layer-7 balancer with path / host routing, TLS termination and an optional WAF. **Front Door** is the global layer-7 entry (sheet 14), and **Traffic Manager** is DNS-based global routing (sheet 13).

KEY POINTS

- Use Standard SKU: Basic Load Balancer and Basic public IPs are retired
- Application Gateway v2 autoscales and is zone-redundant
- Health probes decide which backends receive traffic
- Load Balancer outbound rules can also provide egress

L4 LOAD BALANCER AND L7 APPLICATION GATEWAY

```
az network lb create -g rg-net -n lb-api --sku Standard \
    --frontend-ip-name fe --backend-pool-name be \
    --public-ip-address pip-lb-api
az network lb probe create -g rg-net --lb-name lb-api -n tcp-8080 \
    --protocol Tcp --port 8080
az network lb rule create -g rg-net --lb-name lb-api -n tcp-443 \
    --protocol Tcp --frontend-port 443 --backend-port 8080 \
    --frontend-ip-name fe --backend-pool-name be --probe-name tcp-8080

az network application-gateway create -g rg-net -n agw-shop \
    --sku WAF_v2 --capacity 2 --zones 1 2 3 \
    --vnet-name vnet-shop-prod --subnet snet-agw \
    --public-ip-address pip-agw --priority 100 \
    --servers app-shop.azurewebsites.net --http-settings-protocol Https
az network application-gateway probe create -g rg-net \
    --gateway-name agw-shop -n health --protocol Https \
    --path /health --host-name-from-http-settings true
```

CODE EXPLAINED

- → A Standard LB needs a frontend, a backend pool, a probe and a rule
- → The rule maps port 443 on the frontend to 8080 on the backends
- → Application Gateway WAF_v2 adds web application firewall rules
- → The custom probe checks `/health` over HTTPS

AWS / GCP EQUIVALENT

- ↔ **AWS** NLB (L4), ALB (L7), Global Accelerator, Route 53 (aws-17)
- ↔ **GCP** Network LB, Application LB (regional / global) (gcp-17)

COST CHECK

- $ Standard LB bills per rule-hour plus data processed
- $ Application Gateway v2 bills fixed hourly + capacity units
- $ Use one Application Gateway for many apps via host-based routing

WHICH BALANCER

NEED	USE
TCP / UDP, regional	Azure Load Balancer
HTTP routing, WAF, regional	Application Gateway
Global HTTP + CDN + WAF	Front Door
Global DNS failover	Traffic Manager
Kubernetes ingress	Application Gateway for Containers

LOAD BALANCER PIECES

PIECE	ROLE
Frontend IP	Where clients connect
Backend pool	VMs / scale set / IPs
Health probe	Who is healthy
LB rule	Port mapping
Outbound rule	SNAT for egress

DO & DON'T

- ✓ **Do** Use zone-redundant frontends
- ✓ **Do** Probe a real health endpoint, not just a TCP port, at L7
- ✗ **Don't** Build new solutions on Basic SKUs
- ✗ **Don't** Put Application Gateway in a subnet shared with other resources

LOCAL TESTING

- ⌂ Practice probes and routing with nginx / Traefik in Docker Compose
- ⌂ Network Watcher connection troubleshoot checks backend reachability
- ⌂ `curl -v --resolve host:443:<ip>` tests TLS on the gateway before DNS

PRODUCTION CHECKLIST

- ☐ Health probes tuned with realistic thresholds
- ☐ WAF in Prevention mode on public HTTP endpoints
- ☐ Diagnostics and metrics (unhealthy host count) alerting

Virtual Machine Scale Sets

Flexible vs Uniform orchestration · instance count min / max · autoscale rules · scheduled profiles · upgrade policies · automatic instance repairs

GROUPS OF IDENTICAL VMS

A **Virtual Machine Scale Set (VMSS)** manages a group of VMs from one model and spreads them across zones. **Flexible orchestration** (the default for new sets) uses regular VMs and allows mixing sizes and Spot; Uniform is the older identical-instance mode. **Autoscale** rules add or remove instances on metrics or schedules, and **automatic repairs** replace unhealthy instances.

KEY POINTS

- Upgrade policy decides how model changes roll out: Manual, Automatic or Rolling
- Use a health extension or load balancer probe for repairs and rolling upgrades
- Scheduled profiles scale up before business hours
- Instance protection prevents specific instances from scale-in

LIMITS & QUOTAS

- ≤ Up to 1,000 instances per scale set
- ≤ Autoscale settings: up to 20 profiles each
- ≤ Regional vCPU quotas cap scale-out (sheet 15)

SCALE SET WITH AUTOSCALE

```
az vmss create -g rg-app -n vmss-web --orchestration-mode Flexible \
    --image Ubuntu2404 --vm-sku Standard_D2ps_v6 --instance-count 2 \
    --zones 1 2 3 --platform-fault-domain-count 1 \
    --vnet-name vnet-shop-prod --subnet snet-app \
    --lb lb-api --backend-pool-name be \
    --custom-data cloud-init.yaml --generate-ssh-keys \
    --upgrade-policy-mode Rolling

az monitor autoscale create -g rg-app -n as-web \
    --resource vmss-web \
    --resource-type Microsoft.Compute/virtualMachineScaleSets \
    --min-count 2 --max-count 10 --count 2
az monitor autoscale rule create -g rg-app --autoscale-name as-web \
    --condition "Percentage CPU > 70 avg 5m" --scale out 2
az monitor autoscale rule create -g rg-app --autoscale-name as-web \
    --condition "Percentage CPU < 30 avg 10m" --scale in 1

az vmss update -g rg-app -n vmss-web --enable-automatic-repairs true \
    --automatic-repairs-grace-period 30
az vmss list-instances -g rg-app -n vmss-web -o table
```

CODE EXPLAINED

- → Flexible mode spreads instances across the three zones
- → Rolling upgrades replace instances in batches using health signals
- → Scale out fast (2 at a time), scale in slowly
- → Automatic repairs wait 30 minutes after boot before judging health

AWS / GCP EQUIVALENT

- ↔ **AWS** Launch templates + Auto Scaling groups (aws-18)
- ↔ **GCP** Instance templates + managed instance groups (gcp-18)

COST CHECK

- $ Scale sets are free; you pay for the VMs
- $ Mix Spot and regular instances in Flexible mode for cheap burst capacity
- $ Scheduled scale-in at night saves on predictable workloads

FLEXIBLE VS UNIFORM

ITEM	FLEXIBLE	UNIFORM
Instance type	Regular VMs	Scale-set instances
Mixed sizes / Spot	Yes	No
Zones	Spread across zones	Zone list
Recommended for new	Yes	Legacy / some services

UPGRADE POLICIES

POLICY	BEHAVIOR
Manual	You upgrade instances
Automatic	All at once (risky)
Rolling	Batches with health checks

DO & DON'T

- ✓ **Do** Use Rolling upgrades with a health probe
- ✓ **Do** Keep instances stateless and replaceable
- ✗ **Don't** Scale on CPU alone when queue depth or latency is the real signal
- ✗ **Don't** Set min-count to 1 for production

LOCAL TESTING

- ⌂ Load-test with k6 or Azure Load Testing in a sandbox to tune rules
- ⌂ Test cloud-init locally (sheet 15) before baking into the model
- ⌂ `az monitor autoscale show` reviews rules before go-live

PRODUCTION CHECKLIST

- ☐ Min 2+ instances across zones
- ☐ Automatic repairs and rolling upgrades enabled
- ☐ Autoscale notifications sent to the team

Choosing Compute

VMs vs App Service vs Functions vs Container Apps vs AKS · Static Web Apps · Container Instances · decision table

PICKING THE RIGHT AZURE HOST

Azure offers a host for every level of control. **App Service** runs web apps and APIs from code or containers with slots and autoscale. **Functions** runs event-driven code. **Container Apps** runs containers serverlessly with scale-to-zero, revisions and Dapr. **AKS** gives full Kubernetes. **VMs** give the OS. **Static Web Apps** hosts front ends with optional APIs.

KEY POINTS

- App Service plans are always-on capacity you pay for even when idle
- Container Apps and Functions (Flex / Consumption) scale to zero
- Container Instances run a single container quickly (no orchestration)
- Azure Batch handles large parallel jobs

SAME APP, FOUR HOSTS

```
# App Service: code or container on a plan
az webapp up -n app-shop-api --runtime "PYTHON:3.12" --sku P0v3 -l eastus2

# Container Apps: serverless containers
az containerapp up -n shop-api -g rg-app -l eastus2 \
    --image shopacr.azurecr.io/api:1.4.0 --ingress external --target-port 8080

# Functions: event-driven code
az functionapp create -g rg-app -n func-shop --storage-account stshopfunc \
    --flexconsumption-location eastus2 --runtime python --runtime-version 3.12

# Static Web Apps: front end + optional API
az staticwebapp create -n swa-shop -g rg-app -l eastus2 \
    --source https://github.com/contoso/shop-web --branch main \
    --app-location "/" --output-location "dist" --login-with-github
```

CODE EXPLAINED

- → `az webapp up` creates the plan and app and deploys the current folder
- → `az containerapp up` creates an environment if needed
- → Flex Consumption is the recommended serverless Functions plan
- → Static Web Apps wires a GitHub Actions workflow for you

AWS / GCP EQUIVALENT

- ↔ **AWS** EC2 vs Lambda vs ECS / Fargate vs EKS vs App Runner (aws-19)
- ↔ **GCP** Compute Engine vs Cloud Run vs GKE vs App Engine (gcp-19)

COST CHECK

- $ App Service: fixed hourly per plan instance, many apps can share a plan
- $ Container Apps / Functions Flex: per vCPU-second and memory, free monthly grants
- $ AKS: nodes plus an optional Standard tier fee per cluster

DECISION TABLE

WORKLOAD	BEST FIT
Web app / API, simple ops	App Service or Container Apps
Event processing, glue	Functions
Microservices, scale to zero	Container Apps
Kubernetes ecosystem needs	AKS
Legacy / OS-level control	VMs / scale sets
SPA or static site	Static Web Apps
One-off container task	Container Instances / Container Apps jobs

SCALE TO ZERO

SERVICE	IDLE COST
Functions Flex / Consumption	None (storage only)
Container Apps (min 0)	None
App Service	Plan keeps billing
AKS	Nodes keep billing

DO & DON'T

- ✓ **Do** Start with Container Apps or App Service for HTTP workloads
- ✓ **Do** Containerize so you can move between hosts
- ✗ **Don't** Pick AKS for one small app
- ✗ **Don't** Run one App Service plan per tiny app (share plans)

LOCAL TESTING

- ⌂ Functions Core Tools, Docker and kind cover local runs for each host
- ⌂ Azure Developer CLI (`azd up`) deploys templates to compare hosts quickly (sheet 35)
- ⌂ Static Web Apps CLI: `swa start` emulates routing and auth locally

PRODUCTION CHECKLIST

- □ Host choice documented with expected cost
- □ Autoscale bounds and health checks on the chosen host
- □ Same container image promoted through environments

Sheet 20

Azure Functions Basics

triggers & bindings · Flex Consumption plan · Python v2 / Node.js v4 programming models · `func init` / `new` / `start` · `func azure functionapp publish` · app settings

EVENT-DRIVEN CODE

Azure Functions runs code in response to **triggers** (HTTP, timer, queue, blob, Event Grid, Cosmos DB…) and connects to services with input / output **bindings** instead of SDK boilerplate. The **Flex Consumption** plan is the recommended serverless option: scale to zero, fast scale-out, VNet integration and per-instance memory sizes. The Python v2 and Node.js v4 models declare functions in code with decorators.

KEY POINTS

- A function app is the unit of deploy and scale; it holds many functions
- Each app needs a storage account for its runtime state
- App settings become environment variables
- Use identity-based connections (`__accountName` , `__fullyQualifiedNamespace`) instead of keys

FUNCTION_APP.PY (PYTHON V2 MODEL)

```python
import azure.functions as func, logging, json
app = func.FunctionApp()

@app.route(route="orders", methods=["POST"], auth_level=func.AuthLevel.FUNCTION)
@app.queue_output(arg_name="msg", queue_name="orders",
                  connection="QueueConn")
def create_order(req: func.HttpRequest,
                 msg: func.Out[str]) -> func.HttpResponse:
    order = req.get_json()
    msg.set(json.dumps(order))              # output binding: enqueue
    return func.HttpResponse(status_code=202)

@app.timer_trigger(schedule="0 */15 * * * *", arg_name="t")
def cleanup(t: func.TimerRequest):
    logging.info("cleanup ran")
```

CODE EXPLAINED

- → The HTTP function writes the order to a queue via an output binding
- → Timer schedules use six-field NCRONTAB (with seconds)
- → `func start` runs the same runtime locally
- → The `__queueServiceUri` suffix makes the binding use managed identity

AWS / GCP EQUIVALENT

- ↔ **AWS** Lambda handler, triggers, function URLs (aws-20)
- ↔ **GCP** Cloud Run functions, Eventarc triggers (gcp-21)
- ↔ Bindings have no direct AWS / GCP equivalent

COST CHECK

- $ Flex Consumption: per GB-second and executions, with a monthly free grant
- $ Always-ready instances remove cold starts but bill continuously
- $ The storage account and Application Insights add small costs

BUILD, RUN, PUBLISH

```bash
func init shop-func --python && cd shop-func
func start                                  # local runtime on :7071
az functionapp create -g rg-app -n func-shop --storage-account stshopfunc \
    --flexconsumption-location eastus2 --runtime python --runtime-version 3.12
func azure functionapp publish func-shop
az functionapp config appsettings set -g rg-app -n func-shop \
    --settings QueueConn__queueServiceUri=https://stshop.queue.core.windows.net
```

SECURITY CHECK

- ⚠ Use Entra ID authentication (Easy Auth) or APIM in front, not just function keys
- ⚠ Identity-based connections for storage, Service Bus, Event Hubs
- ⚠ Restrict inbound with private endpoints or access restrictions

LIMITS & QUOTAS

- ≤ Flex Consumption: 30-minute default timeout, can be raised; Consumption: 10-minute max
- ≤ HTTP requests: about 230 seconds before the front end times out
- ≤ Flex instance memory 512 MB, 2 GB or 4 GB; up to 1,000 instances

COMMON TRIGGERS

TRIGGER	FIRES ON
HTTP	Request
Timer	Schedule (NCRONTAB)
Queue / Service Bus	New message
Blob / Event Grid	New or changed blob
Cosmos DB	Change feed
Event Hubs	Stream events

HOSTING PLANS

PLAN	USE
Flex Consumption	Default serverless
Consumption	Legacy serverless (Windows)
Premium (Elastic)	Always warm, long runs
Dedicated (App Service)	Existing plan capacity
Container Apps	Functions in containers

DO & DON'T

- ✓ **Do** Keep functions small and idempotent
- ✓ **Do** Use output bindings to cut SDK code
- ✗ **Don't** Do heavy work inside an HTTP function past the 230-second limit
- ✗ **Don't** Store connection strings with keys in app settings

LOCAL TESTING

- ⌂ `func start` with Azurite for storage triggers
- ⌂ `local.settings.json` holds local app settings (never commit secrets)
- ⌂ VS Code Azure Functions extension debugs locally

PRODUCTION CHECKLIST

- ☐ Identity-based connections and Key Vault references
- ☐ Application Insights with alerts on failures
- ☐ Auth in front (Easy Auth / APIM); function keys not the only control

App Service & Functions Operations

App Service plans · deployment slots & swap · scale up / out · Always On · Durable Functions · cold starts & plan choice

RUNNING WEB APPS IN PRODUCTION

App Service hosts web apps, APIs and Functions on a **plan** (a set of VMs you pay for). **Deployment slots** run staging copies you can **swap** into production with warm-up and instant rollback. Scale **up** to a bigger tier or **out** to more instances, manually or with autoscale. **Durable Functions** adds stateful orchestrations to Functions.

KEY POINTS

- Many apps can share one plan; they share its CPU and memory
- Slot settings marked "deployment slot setting" stay with the slot on swap
- Always On keeps the app loaded (not available on Free / Shared)
- Health check path removes unhealthy instances

LIMITS & QUOTAS

- ≤ Deployment slots: 5 on Standard, 20 on Premium
- ≤ Instances: up to 10 (Standard), 30 (Premium v3)
- ≤ Free / Shared tiers have CPU minutes per day and no slots

PLANS, SLOTS, SWAPS

```
az appservice plan create -g rg-app -n asp-shop --sku P1v3 --is-linux \
    --number-of-workers 2 --zone-redundant
az webapp create -g rg-app -n app-shop-api -p asp-shop \
    --runtime "PYTHON:3.12"
az webapp config set -g rg-app -n app-shop-api --always-on true \
    --health-check-path /health

az webapp deployment slot create -g rg-app -n app-shop-api --slot staging
az webapp deploy -g rg-app -n app-shop-api --slot staging --src-path app.zip
az webapp deployment slot swap -g rg-app -n app-shop-api \
    --slot staging --target-slot production      # swap back to roll back

az appservice plan update -g rg-app -n asp-shop --sku P2v3   # scale up
az webapp log tail -g rg-app -n app-shop-api
```

DURABLE FUNCTIONS ORCHESTRATOR (PYTHON)

```
import azure.durable_functions as df
app = df.DFApp()
@app.orchestration_trigger(context_name="ctx")
def order_flow(ctx: df.DurableOrchestrationContext):
    paid = yield ctx.call_activity("charge", ctx.get_input())
    yield ctx.call_activity("ship", paid)
```

CODE EXPLAINED

- → Zone-redundant plans need at least 2–3 instances
- → Deploy to staging, test, then swap; swapping back is the rollback
- → Scale up changes the tier for every app on the plan
- → The orchestrator replays from history, so it must be deterministic

AWS / GCP EQUIVALENT

- ↔ **AWS** Elastic Beanstalk / App Runner, Lambda aliases, Step Functions (aws-21)
- ↔ **GCP** App Engine versions, Cloud Run revisions, Workflows (gcp-21)

COST CHECK

- $ You pay for plan instances whether apps are busy or idle
- $ Slots run on the same plan: no extra charge, but they use capacity
- $ Premium v3 / v4 often costs less per core than Standard

PLAN TIERS

TIER	USE
Free / Shared	Experiments only
Basic	Dev / test, no slots
Standard	Small prod, 5 slots
Premium v3 / v4	Production default
Isolated v2	Dedicated environment (ASE)

DURABLE PATTERNS

PATTERN	USE
Function chaining	Steps in order
Fan-out / fan-in	Parallel work, then aggregate
Async HTTP API	Long job with status URL
Human interaction	Wait for approval
Monitor	Polling loop

DO & DON'T

- ✓ **Do** Swap through a staging slot with warm-up
- ✓ **Do** Mark per-environment settings as slot settings
- ✗ **Don't** Deploy directly to production slot
- ✗ **Don't** Pack CPU-heavy and latency-sensitive apps on one plan

LOCAL TESTING

- ⌂ `func start` runs Durable Functions locally with Azurite
- ⌂ Run the web app locally in Docker with the same image
- ⌂ `az webapp log tail` streams logs during tests

PRODUCTION CHECKLIST

- ☐ Zone-redundant Premium plan with at least 3 instances
- ☐ Health check path and autoscale rules
- ☐ Slot swap deployments with automated smoke tests

API Management

APIs, operations & products · policies (XML) · subscription keys · `validate-jwt` · rate limits & quotas · tiers (Consumption, v2)

A GATEWAY IN FRONT OF YOUR APIS

Azure API Management (APIM) publishes back-end APIs through a gateway with a developer portal. **APIs** contain operations; **products** bundle APIs for consumers with subscription keys. **Policies** (XML) transform requests and responses, validate JWTs, enforce rate limits and quotas, cache, and route to back ends, at global, product, API or operation scope.

KEY POINTS

- Import from OpenAPI, Functions, App Service or Logic Apps
- Policy pipeline: inbound → backend → outbound → on-error
- Named values hold settings and Key Vault secret references
- v2 tiers deploy in minutes; classic tiers can take much longer

CREATE AND IMPORT

```
az apim create -g rg-api -n apim-shop -l eastus2 --sku-name BasicV2 \
    --publisher-email api@contoso.com --publisher-name Contoso
az apim api import -g rg-api --service-name apim-shop --api-id shop \
    --path shop --specification-format OpenApi \
    --specification-url https://app-shop-api.azurewebsites.net/openapi.json
az apim product create -g rg-api --service-name apim-shop \
    --product-id partners --product-name Partners \
    --subscription-required true --state published
```

CODE EXPLAINED

- → BasicV2 is a quick-to-provision production tier
- → The OpenAPI import creates operations automatically
- → `validate-jwt` rejects requests without a valid Entra ID token
- → The rate limit counts calls per caller IP (key on subscription or user instead if needed)

INBOUND POLICY (XML)

```
<policies>
  <inbound>
    <base />
    <validate-jwt header-name="Authorization" failed-validation-httpcode="401">
      <!-- named value: Entra ID v2.0 .well-known/openid-configuration URL -->
      <openid-config url="{{entra-openid-config-url}}" />
      <audiences><audience>api://shop-api</audience></audiences>
    </validate-jwt>
    <rate-limit-by-key calls="100" renewal-period="60"
        counter-key="@(context.Request.IpAddress)" />
    <set-backend-service base-url="https://app-shop-api.azurewebsites.net" />
  </inbound>
  <backend><base /></backend>
  <outbound><base /></outbound>
  <on-error><base /></on-error>
</policies>
```

AWS / GCP EQUIVALENT

- ↔ **AWS** API Gateway (HTTP / REST APIs), usage plans (aws-22)
- ↔ **GCP** API Gateway, Apigee (gcp-22)
- ↔ APIM policies are closest to Apigee policies

COST CHECK

- $ Consumption: per-call pricing, no fixed fee
- $ v2 tiers: fixed monthly per unit plus overage
- $ Developer tier is cheap but has no SLA: never production

SECURITY CHECK

- ⚠ Validate Entra ID tokens with `validate-jwt`; keys alone identify, they do not authenticate
- ⚠ Lock back ends to APIM only (private networking or managed identity)
- ⚠ Never return back-end error details to clients

TIERS

TIER	USE
Consumption	Serverless, pay per call
Developer	Non-production
Basic v2 / Standard v2	Production, fast deploy
Premium v2	VNet injection, scale
Classic Premium	Multi-region, self-hosted gateways

COMMON POLICIES

POLICY	DOES
validate-jwt	Check tokens
rate-limit-by-key	Throttle
quota-by-key	Volume caps
set-header / rewrite-uri	Transform
cache-lookup / store	Response cache
authentication-managed-identity	Call back end as APIM

DO & DON'T

- ✓ **Do** Keep policies in source control (APIOps)
- ✓ **Do** Use products to separate internal and partner access
- ✗ **Don't** Treat a subscription key as authentication
- ✗ **Don't** Put business logic in policy expressions

LOCAL TESTING

- ⌂ Test policies against a Developer-tier instance in a sandbox
- ⌂ The self-hosted gateway container can run locally against an APIM instance
- ⌂ Validate OpenAPI specs locally with Spectral before import

PRODUCTION CHECKLIST

- ☐ JWT validation on every API
- ☐ Back ends reachable only from APIM
- ☐ Diagnostics to Application Insights; alerts on 4xx / 5xx spikes

Event Grid & Logic Apps

REACTING TO EVENTS AND ORCHESTRATING STEPS

Event Grid routes discrete events from Azure services (system topics) or your apps (custom topics) to handlers such as Functions, queues, webhooks or Event Hubs, with filtering and retries. **Logic Apps** builds workflows with a designer and hundreds of **connectors** (Office 365, Salesforce, SQL, SAP), good for integration without much code.

KEY POINTS

- Event Grid uses push delivery with retry and dead-lettering to storage
- Filters on event type, subject prefix / suffix and advanced fields
- CloudEvents 1.0 is supported alongside the Event Grid schema
- Logic Apps Standard runs single-tenant; Consumption is pay per action

REACT TO BLOB UPLOADS

```
FUNC_ID=$(az functionapp show -g rg-app -n func-shop --query id -o tsv)
az eventgrid system-topic create -g rg-app -n st-uploads-events \
    --location eastus2 --topic-type Microsoft.Storage.StorageAccounts \
    --source $STG
az eventgrid system-topic event-subscription create -g rg-app \
    --system-topic-name st-uploads-events -n on-upload \
    --endpoint-type azurefunction \
    --endpoint "$FUNC_ID/functions/onUpload" \
    --included-event-types Microsoft.Storage.BlobCreated \
    --subject-begins-with /blobServices/default/containers/uploads/ \
    --max-delivery-attempts 10 --event-ttl 1440 \
    --deadletter-endpoint $STG/blobServices/default/containers/deadletter

# Custom topic for your own events
az eventgrid topic create -g rg-app -n egt-orders \
    --input-schema cloudeventschemav1_0
```

CODE EXPLAINED

- → A system topic represents events from one Azure resource
- → Filters limit events to new blobs in the `uploads` container
- → Undeliverable events go to a dead-letter container
- → The Functions trigger receives one event per invocation

EVENT GRID TRIGGER (PYTHON)

```
@app.event_grid_trigger(arg_name="event")
def onUpload(event: func.EventGridEvent):
    url = event.get_json()["url"]           # the new blob
    logging.info("new upload %s", url)
```

AWS / GCP EQUIVALENT

- ↔ **AWS** EventBridge rules, Step Functions (aws-23)
- ↔ **GCP** Eventarc, Workflows, Cloud Scheduler (gcp-23)
- ↔ Logic Apps connectors have no close AWS / GCP twin

COST CHECK

- $ Event Grid: per million operations, first 100,000 free per month
- $ Logic Apps Consumption: per action and connector call
- $ Logic Apps Standard: hosting plan like App Service

WHICH MESSAGING SERVICE

NEED	USE
Discrete events, push	Event Grid
Commands / work queue	Service Bus (sheet 24)
High-volume streams	Event Hubs
MQTT / pull delivery	Event Grid namespaces
Workflow with connectors	Logic Apps
Workflow in code	Durable Functions

LOGIC APPS VS DURABLE FUNCTIONS

ITEM	LOGIC APPS	DURABLE
Authoring	Designer / JSON	Code
Connectors	Hundreds	SDKs
Testing	Harder	Unit tests
Best for	Integration	App logic

DO & DON'T

- ✓ **Do** Filter at the subscription so handlers get only what they need
- ✓ **Do** Configure dead-lettering and alert on it
- ✗ **Don't** Use Event Grid for ordered or transactional messaging
- ✗ **Don't** Build complex app logic in a Logic App designer

LOCAL TESTING

- ⌂ Functions Core Tools can receive Event Grid events via a local tunnel (e.g. VS Code dev tunnels / ngrok)
- ⌂ Post sample events to the local function with `curl` and the Event Grid schema
- ⌂ Logic Apps Standard runs locally in VS Code

PRODUCTION CHECKLIST

- ☐ Dead-letter destinations and retry settings set
- ☐ Handlers idempotent (events can repeat)
- ☐ Metrics on delivery failures alerting

Service Bus & Storage Queues

queues vs topics & subscriptions · peek-lock & settle · sessions · dead-letter queues · Storage Queues · Event Hubs pointer

RELIABLE MESSAGING

Azure Service Bus is the enterprise message broker: **queues** for one consumer group, **topics** with **subscriptions** (each with filters) for fan-out. Receivers use **peek-lock**: a message is locked, processed, then completed, or returned on failure. **Sessions** give ordered processing per key, and failed messages move to a **dead-letter queue**. **Storage Queues** are a simpler, cheaper option.

KEY POINTS

- Peek-lock + complete gives at-least-once processing
- Max delivery count (default 10) before dead-lettering
- Duplicate detection drops repeats by message ID within a window
- Event Hubs is for high-volume streams, not commands

LIMITS & QUOTAS

- ≤ Message size: 256 KB (Standard), up to 100 MB (Premium)
- ≤ Lock duration up to 5 minutes (renew for longer work)
- ≤ Queue / topic size up to 5 GB on Standard; Premium scales with messaging units

NAMESPACE, QUEUE, TOPIC

```
az servicebus namespace create -g rg-msg -n sb-shop -l eastus2 --sku Standard
az servicebus queue create -g rg-msg --namespace-name sb-shop -n orders \
    --max-delivery-count 10 --lock-duration PT1M \
    --enable-dead-lettering-on-message-expiration true
az servicebus topic create -g rg-msg --namespace-name sb-shop -n events
az servicebus topic subscription create -g rg-msg --namespace-name sb-shop \
    --topic-name events -n eu-only
az servicebus topic subscription rule create -g rg-msg \
    --namespace-name sb-shop \
    --topic-name events --subscription-name eu-only -n region \
    --filter-sql-expression "region = 'eu'"
```

CODE EXPLAINED

- → A subscription rule filters topic messages with SQL-like expressions
- → Setting `message_id` enables duplicate detection when turned on
- → `complete_message` removes it; `abandon_message` makes it visible again
- → Roles: Azure Service Bus Data Sender / Data Receiver

SEND AND RECEIVE (PYTHON, MANAGED IDENTITY)

```
from azure.identity import DefaultAzureCredential
from azure.servicebus import ServiceBusClient, ServiceBusMessage
sb = ServiceBusClient("sb-shop.servicebus.windows.net",
                      DefaultAzureCredential())
with sb.get_queue_sender("orders") as s:
    s.send_messages(ServiceBusMessage('{"id": 42}', message_id="42"))
with sb.get_queue_receiver("orders", max_wait_time=5) as r:
    for m in r:
        try:
            process(str(m)); r.complete_message(m)
        except Exception:
            r.abandon_message(m)            # retry until max delivery count
```

AWS / GCP EQUIVALENT

- ↔ **AWS** SQS (queues), SNS (topics), Amazon MQ (aws-24)
- ↔ **GCP** Pub/Sub topics and subscriptions, Cloud Tasks (gcp-24)

COST CHECK

- $ Basic / Standard: per operation; Standard adds a base monthly charge
- $ Premium: per messaging unit per hour, predictable performance
- $ Storage Queues cost a fraction of Service Bus for simple needs

SERVICE BUS VS STORAGE QUEUES

FEATURE	SERVICE BUS	STORAGE QUEUES
Max message	256 KB / 100 MB	64 KB
Topics / filters	Yes	No
Ordering (sessions)	Yes	No
Dead-letter queue	Built in	DIY (poison queue)
Transactions	Yes	No
Cost	Higher	Very low

TIERS

TIER	HAS
Basic	Queues only
Standard	Topics, sessions, transactions
Premium	Isolation, large messages, private endpoints

DO & DON'T

- ✓ **Do** Make consumers idempotent and set message IDs
- ✓ **Do** Alert on dead-letter queue depth
- ✗ **Don't** Hold a lock longer than its duration without renewing
- ✗ **Don't** Put large payloads in messages (store in Blob, send a link)

LOCAL TESTING

- ⌂ The Service Bus emulator runs in Docker with a config file of queues and topics
- ⌂ Azurite emulates Storage Queues
- ⌂ Service Bus Explorer (portal) peeks and resubmits messages

PRODUCTION CHECKLIST

- ☐ Managed identity with Data Sender / Receiver roles
- ☐ Dead-letter monitoring and replay process
- ☐ Premium tier or zone redundancy for critical workloads

Azure Container Registry

`az acr create` · `az acr login` · tag & push · ACR Tasks (`az acr build`) · retention & purge · geo-replication · pull with managed identity

PRIVATE CONTAINER IMAGES

Azure Container Registry (ACR) stores container images and OCI artifacts (Helm charts, signatures). `az acr login` connects Docker, and **ACR Tasks** build images in Azure (`az acr build`) without a local Docker daemon. Pull with a managed identity holding **AcrPull**; Premium adds geo-replication, private endpoints and higher throughput.

KEY POINTS

- Login server: `<name>.azurecr.io`
- Tags are mutable; deploy by digest or use immutable tags (tag locking)
- Purge old images on a schedule with `acr purge`
- Defender for Containers scans images for vulnerabilities

SECURITY CHECK

- ⚠ Disable the admin user; use Entra ID and AcrPull / AcrPush roles
- ⚠ Private endpoint + public access disabled (Premium)
- ⚠ Scan images and block deploys with critical vulnerabilities

LIMITS & QUOTAS

- ≤ Included storage: 10 GiB (Basic), 100 GiB (Standard), 500 GiB (Premium)
- ≤ Geo-replication and private endpoints need Premium
- ≤ Read / write throughput and webhooks scale with tier

CREATE, BUILD, PUSH, PULL

```
az acr create -g rg-app -n shopacr --sku Standard --admin-enabled false
az acr login -n shopacr

docker build -t shopacr.azurecr.io/api:1.4.0 .
docker push shopacr.azurecr.io/api:1.4.0

# Build in Azure from the current folder (no local Docker needed)
az acr build -r shopacr -t api:{{.Run.ID}} -t api:latest .

az acr repository list -n shopacr -o table
az acr repository show-tags -n shopacr --repository api --orderby time_desc

# Lock a release tag against overwrite
az acr repository update -n shopacr --image api:1.4.0 --write-enabled false

# Purge untagged and old images every day
az acr task create -r shopacr -n purge --schedule "0 3 * * *" \
    --cmd "acr purge --filter 'api:.*' --ago 30d --untagged" --context /dev/null

# Let a Container App / AKS pull with managed identity
az role assignment create --assignee-object-id $MI \
    --assignee-principal-type ServicePrincipal --role AcrPull \
    --scope $(az acr show -n shopacr --query id -o tsv)
```

CODE EXPLAINED

- → `az acr login` fetches a token and configures Docker
- → `{{.Run.ID}}` gives each ACR Tasks build a unique tag
- → Disabling writes makes a tag effectively immutable
- → The purge task deletes untagged and month-old images daily

AWS / GCP EQUIVALENT

- ↔ **AWS** ECR, lifecycle policies, image scanning (aws-25)
- ↔ **GCP** Artifact Registry, cleanup policies (gcp-25)

COST CHECK

- $ Fixed daily price per tier plus extra storage per GB
- $ ACR Tasks bill per build-second
- $ Purging old images keeps storage within the included amount

TIERS

FEATURE	BASIC	STANDARD	PREMIUM
Included storage	10 GiB	100 GiB	500 GiB
Geo-replication	No	No	Yes
Private endpoint	No	No	Yes
Content trust / tokens	Limited	Limited	Yes
Best for	Dev	Most prod	Multi-region, private

ROLES

ROLE	CAN
AcrPull	Pull images
AcrPush	Push and pull
AcrDelete	Delete images
Owner / Contributor	Manage registry

DO & DON'T

- ✓ **Do** Tag with version or commit SHA; deploy by digest
- ✓ **Do** Schedule purge tasks
- ✗ **Don't** Enable the admin user for convenience
- ✗ **Don't** Deploy `latest` to production

LOCAL TESTING

- ⌂ Run a local registry: `docker run -d -p 5000:5000 registry:2`
- ⌂ Build with BuildKit locally; push to ACR only from CI
- ⌂ Trivy scans images locally before push

PRODUCTION CHECKLIST

- ☐ Admin user disabled; AcrPull via managed identity
- ☐ Vulnerability scanning with deploy gates
- ☐ Premium with private endpoint and geo-replication where needed

Azure Container Apps

environments · revisions & traffic splitting · ingress · scale rules (KEDA) · Dapr · jobs · `az containerapp up`

SERVERLESS CONTAINERS

Azure Container Apps runs containers without managing Kubernetes. Apps live in an **environment** (a shared network and logging boundary). Each deploy creates a **revision**; in multiple-revision mode you can split traffic for canary releases. **Scale rules** (powered by KEDA) scale on HTTP concurrency, queues, events or CPU, including to zero. **Jobs** run scheduled or event-driven tasks.

KEY POINTS

- Ingress can be external or internal to the environment
- Built-in Dapr adds service invocation, pub/sub and state APIs
- Secrets can reference Key Vault via managed identity
- Workload profiles add dedicated hardware options to the consumption model

SECURITY CHECK

- ⚠ Use internal ingress for back-end services
- ⚠ Pull images and read secrets with managed identity (no registry passwords)
- ⚠ Deploy into your own VNet for private networking

LIMITS & QUOTAS

- ≤ Up to 1,000 replicas per revision
- ≤ Consumption replicas: up to 4 vCPU / 8 GiB each
- ≤ Environment core quotas per region; request increases for large fleets

ENVIRONMENT, APP, REVISIONS, JOB

```
az containerapp env create -g rg-app -n cae-shop -l eastus2 \
    --infrastructure-subnet-resource-id $SUBNET_ACA

az containerapp create -g rg-app -n api --environment cae-shop \
    --image shopacr.azurecr.io/api:1.4.0 \
    --registry-server shopacr.azurecr.io --registry-identity $MI_ID \
    --user-assigned $MI_ID --ingress external --target-port 8080 \
    --min-replicas 0 --max-replicas 20 \
    --scale-rule-name http --scale-rule-type http \
    --scale-rule-http-concurrency 50 \
    --secrets "db-pass=keyvaultref:$SECRET_URI,identityref:$MI_ID" \
    --env-vars DB_PASSWORD=secretref:db-pass

# Canary: two revisions, 90/10 split
az containerapp revision set-mode -g rg-app -n api --mode multiple
az containerapp update -g rg-app -n api --image shopacr.azurecr.io/api:1.5.0
az containerapp ingress traffic set -g rg-app -n api \
    --revision-weight api--v14=90 latest=10

# Scheduled job
az containerapp job create -g rg-app -n nightly --environment cae-shop \
    --trigger-type Schedule --cron-expression "0 2 * * *" \
    --image shopacr.azurecr.io/batch:1.0 --replica-timeout 1800
```

CODE EXPLAINED

- → The environment joins your VNet through a delegated subnet
- → Managed identity pulls the image and resolves the Key Vault secret
- → HTTP concurrency of 50 adds a replica per 50 concurrent requests
- → Traffic weights shift users between revisions gradually

AWS / GCP EQUIVALENT

- ↔ **AWS** ECS on Fargate, App Runner (aws-26)
- ↔ **GCP** Cloud Run services and jobs (gcp-20, gcp-21)
- ↔ Container Apps ≈ Cloud Run with built-in Dapr and KEDA

COST CHECK

- $ Consumption: per vCPU-second and GiB-second with a monthly free grant
- $ Scale to zero means no compute charge when idle
- $ Dedicated workload profiles bill per node like VMs

SCALE RULES

RULE	SCALES ON
`http`	Concurrent requests
`tcp`	Concurrent connections
`azure-servicebus`	Queue length
`azure-queue`	Storage queue length
`custom (KEDA)`	Any KEDA scaler
`cpu / memory`	Utilization (not to zero)

COMMANDS

COMMAND	DOES
`az containerapp up`	Build / create in one step
`az containerapp logs show --follow`	Stream logs
`az containerapp exec`	Shell into a replica
`az containerapp revision list`	Revisions
`az containerapp job start`	Run a job now

DO & DON'T

- ✓ **Do** Use multiple-revision mode for canaries and instant rollback
- ✓ **Do** Keep min replicas at 1 for latency-sensitive APIs
- ✗ **Don't** Store registry passwords as secrets when managed identity works
- ✗ **Don't** Expose internal services with external ingress

LOCAL TESTING

- ⌂ Run the same image with `docker run -p 8080:8080`
- ⌂ Dapr CLI: `dapr run` emulates sidecar APIs locally
- ⌂ Docker Compose for multi-container local stacks

PRODUCTION CHECKLIST

- ☐ Managed identity for registry and secrets
- ☐ VNet-integrated environment with internal ingress for back ends
- ☐ Health probes, min replicas and alerts set

AKS

MANAGED KUBERNETES ON AZURE

Azure Kubernetes Service (AKS) runs the Kubernetes control plane for you; you choose node pools, networking and add-ons. **AKS Automatic** goes further, managing nodes, scaling, upgrades and security defaults. **Workload identity** lets pods use Entra ID identities through federated credentials. Attaching ACR lets nodes pull images without secrets.

KEY POINTS

- System node pool runs cluster add-ons; user pools run apps
- Azure CNI Overlay is the common default network mode
- Upgrades: control plane first, then node pools (auto-upgrade channels available)
- See the Kubernetes set for kubectl and manifests

CLUSTER, POOLS, IDENTITY

```
az aks create -g rg-aks -n aks-shop -l eastus2 --tier standard \
    --node-count 3 --zones 1 2 3 --node-vm-size Standard_D4ps_v6 \
    --network-plugin azure --network-plugin-mode overlay \
    --enable-oidc-issuer --enable-workload-identity \
    --attach-acr shopacr --generate-ssh-keys
# Or let Azure manage nodes and defaults
az aks create -g rg-aks -n aks-auto --sku automatic -l eastus2

az aks get-credentials -g rg-aks -n aks-shop
kubectl get nodes -o wide

az aks nodepool add -g rg-aks --cluster-name aks-shop -n spot \
    --priority Spot --eviction-policy Delete --spot-max-price -1 \
    --enable-cluster-autoscaler --min-count 0 --max-count 10

# Workload identity: federate a Kubernetes service account
ISSUER=$(az aks show -g rg-aks -n aks-shop \
    --query oidcIssuerProfile.issuerUrl -o tsv)
az identity federated-credential create -g rg-aks --identity-name id-api \
    -n api-sa --issuer $ISSUER \
    --subject system:serviceaccount:shop:api \
    --audiences api://AzureADTokenExchange

az aks get-upgrades -g rg-aks -n aks-shop -o table
az aks upgrade -g rg-aks -n aks-shop --kubernetes-version <version>
```

CODE EXPLAINED

- → Standard tier adds the uptime SLA and larger scale limits
- → `--attach-acr` grants the kubelet identity AcrPull
- → A Spot pool scales from zero for cheap batch work
- → Pods using service account `shop/api` get the identity's Entra tokens

AWS / GCP EQUIVALENT

- ↔ **AWS** EKS, eksctl, Pod Identity, EKS Auto Mode (aws-27)
- ↔ **GCP** GKE Standard / Autopilot, Workload Identity Federation (gcp-26, gcp-27)

COST CHECK

- $ Free tier: no cluster fee, no SLA; Standard tier: hourly fee per cluster
- $ You pay for node VMs, disks and load balancers
- $ Spot pools and the cluster autoscaler cut node cost

LIMITS & QUOTAS

- ≤ Up to 5,000 nodes per cluster (Standard tier)
- ≤ Up to 100 node pools per cluster
- ≤ Up to 250 pods per node (set at pool creation)

TIERS & MODES

OPTION	USE
Free tier	Dev / test
Standard tier	Production with SLA
Premium tier	Long-term support versions
AKS Automatic	Managed nodes and defaults
AKS Standard (manual)	Full control of pools

COMMON ADD-ONS

ADD-ON	DOES
Azure Monitor (Container insights)	Logs and metrics
Managed Prometheus / Grafana	Metrics stack
Key Vault CSI driver	Mount secrets
App routing	Managed NGINX ingress
Azure Policy	Pod security guardrails
KEDA	Event-driven autoscaling

DO & DON'T

- ✓ **Do** Use workload identity for pods; never mount cloud secrets as env vars by hand
- ✓ **Do** Spread node pools across zones
- ✗ **Don't** Run apps on the system node pool
- ✗ **Don't** Fall far behind on Kubernetes versions (support windows are short)

LOCAL TESTING

- ⌂ kind / k3d / minikube test the same manifests locally
- ⌂ Bridge to Kubernetes / mirrord connect local code to a cluster
- ⌂ `az aks command invoke` runs kubectl against private clusters

PRODUCTION CHECKLIST

- ☐ Standard tier, three zones, autoscaler bounds set
- ☐ Auto-upgrade channel and maintenance windows configured
- ☐ Workload identity, Key Vault CSI and Azure Policy enabled

Blob Storage Basics

OBJECT STORAGE ON AZURE

Blob Storage keeps objects (**blobs**) in **containers** inside a **storage account**, the unit that sets the name, region, redundancy and network rules. Access tiers trade storage price for read price: **Hot**, **Cool**, **Cold** and **Archive**. Redundancy ranges from three copies in one datacenter (LRS) to zone (ZRS) and cross-region (GRS / GZRS) replication.

KEY POINTS

- Endpoint: `https://<account>.blob.core.windows.net/<container>/<blob>`
- Use `--auth-mode login` so the CLI uses Entra ID, not account keys
- AzCopy is the fast tool for bulk copy and sync
- Archive blobs must be rehydrated (hours) before reading

SECURITY CHECK

- ⚠ Create accounts with anonymous access and shared keys disabled
- ⚠ Require TLS 1.2+ and HTTPS only
- ⚠ Use private endpoints or network rules for production accounts

LIMITS & QUOTAS

- ≤ Default max capacity 5 PiB per account
- ≤ 250 storage accounts per region per subscription (can be raised)
- ≤ Single block blob up to about 190.7 TiB

ACCOUNT, CONTAINER, COPY

```
az storage account create -g rg-data -n stshopdata -l eastus2 \
    --sku Standard_ZRS --kind StorageV2 --min-tls-version TLS1_2 \
    --allow-blob-public-access false --allow-shared-key-access false
az role assignment create --assignee $ME \
    --role "Storage Blob Data Contributor" \
    --scope $(az storage account show -n stshopdata --query id -o tsv)

az storage container create --account-name stshopdata -n uploads \
    --auth-mode login
az storage blob upload --account-name stshopdata -c uploads \
    -f photo.jpg -n 2026/11/photo.jpg --auth-mode login
az storage blob upload-batch --account-name stshopdata -d uploads \
    -s ./images --pattern "*.jpg" --auth-mode login
az storage blob list --account-name stshopdata -c uploads \
    --prefix 2026/ --auth-mode login -o table
az storage blob set-tier --account-name stshopdata -c uploads \
    -n 2026/11/photo.jpg --tier Cool --auth-mode login

azcopy login
azcopy sync ./site "https://stshopdata.blob.core.windows.net/site" \
    --delete-destination=true
```

CODE EXPLAINED

- → ZRS keeps copies in three zones of the region
- → With shared keys off, you need a data role (Blob Data Contributor)
- → `upload-batch` uploads a folder by pattern
- → AzCopy `sync` mirrors a folder and can delete extra blobs

AWS / GCP EQUIVALENT

- ↔ **AWS** S3 buckets and storage classes (aws-28)
- ↔ **GCP** Cloud Storage buckets and classes (gcp-28)
- ↔ Azure adds the storage account layer above containers

COST CHECK

- $ Billed for capacity per tier, operations, retrieval (cool / cold / archive) and egress
- $ Early deletion fees: Cool 30, Cold 90, Archive 180 days
- $ GRS roughly doubles storage price versus LRS

TIERS

TIER	USE	MIN DAYS
Hot	Frequent access	–
Cool	Monthly access	30
Cold	Quarterly access	90
Archive	Rare, offline	180

REDUNDANCY

SKU	COPIES
LRS	3 in one datacenter
ZRS	3 across zones
GRS	LRS + async copy to paired region
GZRS	ZRS + async copy to paired region
RA-GRS / RA-GZRS	Plus read access in secondary

DO & DON'T

- ✓ **Do** One account per app / environment with clear names
- ✓ **Do** Use lifecycle rules to move old data to colder tiers (sheet 29)
- ✗ **Don't** Hand out account keys or connection strings
- ✗ **Don't** Read archive data frequently (rehydration fees and delays)

LOCAL TESTING

- ⌂ Azurite emulates Blob, Queue and Table storage: `docker run -p 10000:10000 mcr.microsoft.com/azure-storage/azurite`
- ⌂ Azure Storage Explorer browses Azurite and real accounts
- ⌂ Connection string `UseDevelopmentStorage=true` targets Azurite

PRODUCTION CHECKLIST

- ☐ Anonymous access and shared keys disabled
- ☐ ZRS or GZRS for production data
- ☐ Private endpoint, soft delete and versioning enabled

Storage Security & Features

Entra ID data roles · SAS tokens (user delegation SAS) · disable shared key & anonymous access · versioning & soft delete · lifecycle management · static website hosting

PROTECTING AND MANAGING BLOB DATA

Grant access with Entra ID **data roles**; when a client needs a temporary link, issue a **user delegation SAS** signed with Entra credentials rather than account keys. **Versioning**, **soft delete** and **immutability** protect against overwrites, deletes and ransomware. **Lifecycle management** moves or deletes blobs by age, and **static website** hosting serves a `$web` container.

KEY POINTS

- Account-key SAS tokens cannot be revoked without rotating the key
- User delegation SAS lasts up to 7 days and can be revoked by revoking delegation keys
- Point-in-time restore for block blobs builds on versioning and change feed
- Immutability policies make blobs write-once (WORM)

SECURITY CHECK

- ⚠ Prefer user delegation SAS with short expiry and minimal permissions
- ⚠ Turn on versioning and soft delete for blobs and containers
- ⚠ Alert on any change that re-enables public or shared-key access

SAS, PROTECTION, LIFECYCLE, STATIC SITE

```
az storage blob generate-sas --account-name stshopdata -c uploads \
    -n 2026/11/photo.jpg --permissions r --expiry 2026-11-13T00:00Z \
    --as-user --auth-mode login --full-uri -o tsv

az storage account blob-service-properties update -n stshopdata -g rg-data \
    --enable-versioning true --enable-delete-retention true \
    --delete-retention-days 14 --enable-container-delete-retention true \
    --container-delete-retention-days 14

az storage account management-policy create -n stshopdata -g rg-data \
    --policy @lifecycle.json

az storage blob service-properties update --account-name stshopdata \
    --static-website --index-document index.html --404-document 404.html \
    --auth-mode login
az storage account show -n stshopdata --query primaryEndpoints.web -o tsv
```

CODE EXPLAINED

- → `--as-user` signs the SAS with a user delegation key from Entra ID
- → Soft delete keeps deleted blobs and containers for 14 days
- → The lifecycle rule cools logs at 30 days, archives at 180, deletes at 2 years
- → Static website content goes in the `$web` container

AWS / GCP EQUIVALENT

- ↔ **AWS** S3 bucket policies, presigned URLs, versioning, lifecycle (aws-29)
- ↔ **GCP** IAM, signed URLs, versioning, soft delete, lifecycle (gcp-29)

LIFECYCLE.JSON

```
{ "rules": [{ "name": "age-out", "enabled": true, "type": "Lifecycle",
  "definition": {
    "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["logs/"] },
    "actions": { "baseBlob": {
      "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
      "tierToArchive": { "daysAfterModificationGreaterThan": 180 },
      "delete":        { "daysAfterModificationGreaterThan": 730 } } } } }] }
```

COST CHECK

- $ Versions and soft-deleted blobs bill as normal storage: pair with lifecycle cleanup
- $ Lifecycle actions are free; tier changes bill transaction fees
- $ Static website hosting is cheap; add Front Door for HTTPS on custom domains

ACCESS OPTIONS

METHOD	USE
Entra ID + data role	Apps and users (default)
User delegation SAS	Temporary link, Entra-signed
Service / account SAS	Legacy; key-signed
Anonymous read	Avoid; use CDN instead
Account key	Disable

DATA PROTECTION

FEATURE	PROTECTS AGAINST
Soft delete (blob / container)	Accidental delete
Versioning	Overwrites
Point-in-time restore	Bulk mistakes
Immutability (WORM)	Tampering, ransomware
Resource lock	Account deletion

DO & DON'T

- ✓ **Do** Issue SAS links valid for minutes or hours, read-only
- ✓ **Do** Pair versioning with lifecycle rules that delete old versions
- ✗ **Don't** Embed long-lived account-key SAS tokens in apps
- ✗ **Don't** Serve a public site straight from a public container

LOCAL TESTING

- ⌂ Azurite supports SAS generation and soft delete for testing
- ⌂ Validate lifecycle JSON with `az storage account management-policy show` in a sandbox
- ⌂ Test static sites locally with `npx serve`

PRODUCTION CHECKLIST

- ☐ Versioning, soft delete and lifecycle cleanup configured
- ☐ Only user delegation SAS allowed (policy / key access disabled)
- ☐ Front Door with HTTPS in front of static websites

Disks, Files & Backup

managed disk types (Premium SSD v2, Standard SSD) · snapshots · Azure Files (SMB, NFS) · Blob vs Disks vs Files · Azure Backup & Recovery Services vaults

BLOCK, FILE AND BACKUP STORAGE

Managed disks are block devices attached to VMs, sized and tiered independently. **Azure Files** provides fully managed SMB and NFS file shares that many VMs, containers or on-premises machines can mount. **Azure Backup** protects VMs, disks, file shares, SQL and blobs with policies stored in **Recovery Services** or **Backup** vaults.

KEY POINTS

- Premium SSD v2 lets you set IOPS and throughput separately from size
- Azure Files SMB works from Windows, macOS and Linux; NFS needs Premium
- Azure File Sync caches shares on Windows servers
- Backup vaults support soft delete and immutability

FILE SHARE AND VM BACKUP

```
# Azure Files: share and mount (Linux, SMB)
az storage share-rm create -g rg-data --storage-account stshopfiles \
    -n shared --quota 100 --enabled-protocols SMB
sudo mount -t cifs //stshopfiles.file.core.windows.net/shared /mnt/shared \
    -o vers=3.1.1,credentials=/etc/smbcredentials/stshopfiles.cred,serverino

# Disk snapshot and restore to a new disk
az snapshot create -g rg-app -n snap-data --source disk-data --incremental true
az disk create -g rg-app -n disk-data-restored --source snap-data

# Azure Backup for a VM
az backup vault create -g rg-backup -n rsv-shop -l eastus2
az backup protection enable-for-vm -g rg-backup --vault-name rsv-shop \
    --vm $(az vm show -g rg-app -n vm-web-1 --query id -o tsv) \
    --policy-name DefaultPolicy
az backup job list -g rg-backup --vault-name rsv-shop -o table
```

CODE EXPLAINED

- → `share-rm` creates the share through Resource Manager
- → SMB 3.1.1 with encryption is required over the internet
- → A disk can be created straight from a snapshot
- → DefaultPolicy backs up daily and keeps 30 days

AWS / GCP EQUIVALENT

- ↔ **AWS** EBS, EFS, AWS Backup (aws-30)
- ↔ **GCP** Persistent Disk / Hyperdisk, Filestore, Backup and DR (gcp-30)

COST CHECK

- $ Disks bill by provisioned size (and IOPS / throughput for v2), not usage
- $ Azure Files: by provisioned or used capacity depending on billing model
- $ Backup: per protected instance plus vault storage

WHICH STORAGE

NEED	USE
Objects over HTTPS	Blob Storage
VM boot / data disk	Managed disks
Shared SMB / NFS folder	Azure Files
High-performance NFS	Azure NetApp Files
Large-scale HPC	Azure Managed Lustre

BACKUP TARGETS

WORKLOAD	VAULT
Azure VMs	Recovery Services
Azure Files	Recovery Services
SQL / SAP HANA in VMs	Recovery Services
Disks	Backup vault
Blobs	Backup vault
PostgreSQL Flexible	Backup vault (long-term)

DO & DON'T

- ✓ **Do** Use Premium SSD v2 to tune IOPS without oversizing disks
- ✓ **Do** Enable soft delete and immutability on backup vaults
- ✗ **Don't** Treat snapshots as your only backup
- ✗ **Don't** Mount SMB shares with account keys stored in plain files on many servers

LOCAL TESTING

- ⌂ Practice SMB mounts against a local Samba container
- ⌂ Restore drills: restore a VM backup to a new resource group
- ⌂ fio benchmarks disk IOPS on a test VM

PRODUCTION CHECKLIST

- ☐ Backup policies on every production VM and file share
- ☐ Restore tested on a schedule
- ☐ Vault soft delete, immutability and cross-region restore where required

Azure SQL & PostgreSQL

Azure SQL Database vs Managed Instance · serverless & Hyperscale · Azure Database for PostgreSQL / MySQL Flexible Server · high availability · point-in-time restore · Entra authentication

MANAGED RELATIONAL DATABASES

Azure SQL Database is managed SQL Server with **serverless** (auto-pause, per-second billing) and **Hyperscale** (very large, fast scaling) options; **SQL Managed Instance** offers near-full SQL Server compatibility. **Azure Database for PostgreSQL / MySQL Flexible Server** runs the open-source engines with zone-redundant high availability, read replicas and **point-in-time restore**.

KEY POINTS

- Use Entra authentication and managed identities instead of SQL passwords
- Zone-redundant HA keeps a standby in another zone
- PITR restores to a new server or database
- Private endpoints / VNet integration keep databases off the internet

AZURE SQL (SERVERLESS) AND POSTGRESQL FLEXIBLE SERVER

```
az sql server create -g rg-data -n sql-shop -l eastus2 \
    --enable-ad-only-auth --external-admin-principal-type Group \
    --external-admin-name sg-dba --external-admin-sid <group-object-id>
az sql db create -g rg-data -s sql-shop -n shopdb \
    --edition GeneralPurpose --compute-model Serverless --family Gen5 \
    --capacity 2 --auto-pause-delay 60 --backup-storage-redundancy Zone

az postgres flexible-server create -g rg-data -n pg-shop -l eastus2 \
    --tier GeneralPurpose --sku-name Standard_D2ds_v5 --version 17 \
    --storage-size 128 --high-availability ZoneRedundant \
    --public-access None --microsoft-entra-auth Enabled --password-auth Disabled
az postgres flexible-server replica create -g rg-data \
    --replica-name pg-shop-read --source-server pg-shop

az postgres flexible-server restore -g rg-data -n pg-shop-restored \
    --source-server pg-shop --restore-time "2026-11-12T08:00:00Z"
```

CODE EXPLAINED

- → Entra-only auth means no SQL logins with passwords
- → Serverless pauses after 60 idle minutes and bills storage only
- → Zone-redundant HA and a read replica for scale-out reads
- → The Entra access token is used as the PostgreSQL password

AWS / GCP EQUIVALENT

- ↔ **AWS** RDS / Aurora, RDS Proxy (aws-31)
- ↔ **GCP** Cloud SQL, AlloyDB, Cloud SQL Auth Proxy (gcp-31)

COST CHECK

- $ Serverless SQL bills per vCore-second while active; good for intermittent use
- $ Zone-redundant HA roughly doubles compute cost
- $ Reserved capacity discounts steady databases

CONNECT WITH AN ENTRA TOKEN (PYTHON)

```
import psycopg
from azure.identity import DefaultAzureCredential
tok = DefaultAzureCredential().get_token(
    "https://ossrdbms-aad.database.windows.net/.default").token
conn = psycopg.connect(host="pg-shop.postgres.database.azure.com",
                       dbname="shop", user="app-identity", password=tok,
                       sslmode="require")
```

SECURITY CHECK

- ⚠ Enable Entra-only authentication where possible
- ⚠ Disable public network access; use private endpoints
- ⚠ Microsoft Defender for SQL / open-source databases for threat detection

LIMITS & QUOTAS

- ≤ Point-in-time restore window: 1–35 days
- ≤ Azure SQL General Purpose up to 4 TB; Hyperscale up to 128 TB
- ≤ Flexible Server max connections depend on SKU

OPTIONS

SERVICE	USE
Azure SQL Database	New SQL Server apps
Azure SQL Hyperscale	Large or fast-growing DBs
SQL Managed Instance	Lift-and-shift SQL Server
PostgreSQL Flexible Server	Open-source default
MySQL Flexible Server	MySQL apps
SQL Server on VMs	Full control / legacy

PURCHASING MODELS

MODEL	NOTES
vCore provisioned	Pick cores, reserve for discount
vCore serverless	Auto-pause, per second
DTU	Older bundled model
Flexible Server tiers	Burstable, General Purpose, Memory Optimized

DO & DON'T

- ✓ **Do** Connect apps with managed identity tokens
- ✓ **Do** Test PITR restores regularly
- ✗ **Don't** Expose databases with "Allow Azure services" and public access
- ✗ **Don't** Pick Burstable tiers for production with HA needs

LOCAL TESTING

- ⌂ Run SQL Server or PostgreSQL in Docker with the same version
- ⌂ Azure SQL Edge / SQL Server containers for local T-SQL work
- ⌂ Testcontainers for integration tests

PRODUCTION CHECKLIST

- □ Entra auth, private endpoints, public access off
- □ Zone-redundant HA and backups with suitable retention
- □ Query Performance Insight / Query Store and alerts on DTU / CPU

Cosmos DB

GLOBALLY DISTRIBUTED NOSQL

Azure Cosmos DB stores JSON items in **containers** spread across physical partitions by a **partition key**. Every operation costs **request units (RUs)**; you buy throughput as provisioned, **autoscale** or **serverless**. Five **consistency levels** trade latency for freshness, and the **change feed** streams every write. Multiple APIs exist: NoSQL (native), MongoDB, Cassandra, Gremlin and Table.

KEY POINTS

- Pick a partition key with high cardinality that appears in most queries
- A point read of a 1 KB item by id + partition key costs 1 RU
- Cross-partition queries cost more and scale worse
- Session consistency is the default and fits most apps

ACCOUNT, CONTAINER, DATA ROLE

```
az cosmosdb create -g rg-data -n cosmos-shop \
    --default-consistency-level Session \
    --locations regionName=eastus2 failoverPriority=0 isZoneRedundant=True
COSMOS=$(az cosmosdb show -g rg-data -n cosmos-shop --query id -o tsv)
az resource update --ids $COSMOS \
    --set properties.disableLocalAuth=true
az cosmosdb sql database create -g rg-data -a cosmos-shop -n shop
az cosmosdb sql container create -g rg-data -a cosmos-shop -d shop \
    -n orders --partition-key-path /customerId --max-throughput 4000

# Built-in Data Contributor role for the app identity
az cosmosdb sql role assignment create -g rg-data -a cosmos-shop \
    --role-definition-id 00000000-0000-0000-0000-000000000002 \
    --principal-id $MI --scope "/"
```

CODE EXPLAINED

- → Zone redundancy is set per region location
- → `disableLocalAuth` turns off account keys
- → Autoscale scales between 10% and 100% of 4,000 RU/s
- → Passing `partition_key` keeps queries to one partition

AWS / GCP EQUIVALENT

- ↔ **AWS** DynamoDB (partition / sort keys, RCU / WCU) (aws-32)
- ↔ **GCP** Firestore, Spanner, Bigtable (gcp-32)
- ↔ RUs ≈ DynamoDB capacity units; partition key choice matters the same way

COST CHECK

- $ Provisioned: per 100 RU/s per hour; autoscale bills the hourly peak
- $ Serverless: per million RUs consumed, best for spiky low traffic
- $ Storage per GB per region; every extra region multiplies cost

SECURITY CHECK

- ⚠ Disable key-based (local) auth; use Entra data-plane RBAC
- ⚠ Private endpoints with public network access off
- ⚠ Least-privilege data roles (Reader vs Contributor)

LIMITS & QUOTAS

- ≤ Item size up to 2 MB
- ≤ Logical partition up to 20 GB
- ≤ Physical partition up to 10,000 RU/s

PYTHON SDK

```
from azure.cosmos import CosmosClient
from azure.identity import DefaultAzureCredential
db = CosmosClient("https://cosmos-shop.documents.azure.com",
                  DefaultAzureCredential()).get_database_client("shop")
orders = db.get_container_client("orders")
orders.upsert_item({"id": "o-42", "customerId": "c-7", "total": 59.0})
item = orders.read_item("o-42", partition_key="c-7")       # ~1 RU
for o in orders.query_items("SELECT * FROM o WHERE o.total > @t",
        parameters=[{"name": "@t", "value": 50}], partition_key="c-7"):
    print(o["id"])
```

CONSISTENCY LEVELS

LEVEL	GUARANTEE
Strong	Latest committed write
Bounded staleness	Lag within K versions or T time
Session (default)	Read your own writes
Consistent prefix	No out-of-order reads
Eventual	Lowest latency, no ordering

THROUGHPUT MODES

MODE	BEST FOR
Provisioned (manual)	Steady, predictable load
Autoscale	Variable load with a peak
Serverless	Dev / spiky, low traffic

DO & DON'T

- ✓ **Do** Model around access patterns and the partition key first
- ✓ **Do** Use point reads (id + partition key) whenever possible
- ✗ **Don't** Choose a low-cardinality partition key like `country`
- ✗ **Don't** Run unbounded cross-partition queries on hot paths

LOCAL TESTING

- ⌂ The Cosmos DB emulator runs on Windows or as a Linux Docker image
- ⌂ Point the SDK at the emulator endpoint with its well-known key
- ⌂ Check RU charge per query in the response headers while testing

PRODUCTION CHECKLIST

- □ Local auth disabled; Entra data roles only
- □ Zone redundancy and, if needed, multi-region with failover
- □ Alerts on 429 throttling and RU consumption

Redis & Other Data Services

CACHES, SEARCH AND ANALYTICS

Azure Managed Redis is the current Redis offering (built on Redis Enterprise); **Azure Cache for Redis** is being retired, so new work should start on Managed Redis. **Azure AI Search** provides full-text, vector and hybrid search, often the retrieval layer for RAG. **Microsoft Fabric** (and older Synapse) covers analytics and data warehousing.

KEY POINTS

- Cache-aside: read cache, fall back to the database, then populate
- Managed Redis supports Entra ID authentication
- AI Search indexes data from Blob, SQL, Cosmos DB and more
- See the SQL and NoSQL sets for engine details

MANAGED REDIS, AI SEARCH

```
az redisenterprise create -g rg-data -n amr-shop -l eastus2 \
    --sku Balanced_B1
az redisenterprise database list -g rg-data --cluster-name amr-shop -o table

az search service create -g rg-search -n srch-shop --sku basic \
    --partition-count 1 --replica-count 2 --disable-local-auth true
```

CACHE-ASIDE IN PYTHON

```
import json, redis
from redis_entraid.cred_provider import create_from_default_azure_credential
creds = create_from_default_azure_credential(
    ("https://redis.azure.com/.default",))
r = redis.Redis(host="amr-shop.eastus2.redis.azure.net", port=10000,
                ssl=True, credential_provider=creds)

def get_product(pid):
    if (hit := r.get(f"product:{pid}")):
        return json.loads(hit)
    row = db_fetch_product(pid)                  # slow path
    r.set(f"product:{pid}", json.dumps(row), ex=300)
    return row
```

CODE EXPLAINED

- → Managed Redis is managed with the `redisenterprise` CLI group
- → AI Search with 2 replicas gets a read SLA
- → The `redis-entraid` package supplies Entra tokens to redis-py
- → Cache entries expire after 300 seconds

AWS / GCP EQUIVALENT

- ↔ **AWS** ElastiCache (Valkey / Redis OSS), OpenSearch (aws-33)
- ↔ **GCP** Memorystore (Valkey / Redis), BigQuery, Vertex AI Search (gcp-33)

COST CHECK

- $ Managed Redis bills per hour by SKU; size the memory to the working set
- $ AI Search: per search unit per hour (partitions × replicas)
- $ Fabric bills by capacity; pause when idle

CHOOSING A DATA SERVICE

NEED	USE
Relational, SQL Server	Azure SQL (sheet 31)
Relational, open source	PostgreSQL Flexible (sheet 31)
Global NoSQL / document	Cosmos DB (sheet 32)
Cache / session store	Azure Managed Redis
Full-text / vector search	Azure AI Search
Analytics / warehouse	Microsoft Fabric
Time series / logs	Azure Data Explorer

REDIS OPTIONS

OPTION	STATUS
Azure Managed Redis	Current, recommended
Azure Cache for Redis	Retiring; migrate
Redis on AKS / VMs	Self-managed

DO & DON'T

- ✓ **Do** Set TTLs on every cache key
- ✓ **Do** Plan migration from Azure Cache for Redis to Managed Redis
- ✗ **Don't** Treat a cache as the source of truth
- ✗ **Don't** Index everything in AI Search without a retrieval plan

LOCAL TESTING

- ⌂ Run Redis or Valkey in Docker: `docker run -p 6379:6379 redis`
- ⌂ Elasticsearch / OpenSearch containers approximate search behavior locally
- ⌂ Mock the cache layer in unit tests

PRODUCTION CHECKLIST

- ☐ Cache with zone redundancy and Entra auth
- ☐ AI Search with 2+ replicas and private endpoint
- ☐ Cache hit-rate and memory alerts

Bicep

resources, parameters & outputs · modules · `az deployment group create` · `what-if` · deployment scopes · deployment stacks · ARM JSON relationship

AZURE'S NATIVE IAC LANGUAGE

Bicep is a concise language that compiles to ARM JSON templates. A file declares **parameters**, **resources** (with API versions), and **outputs**; **modules** reuse other Bicep files. Deploy with `az deployment group create` at resource group scope (or subscription, management group, tenant), preview with **what-if**, and use **deployment stacks** to manage and protect the resources a template owns.

KEY POINTS

- Resource references create dependencies automatically
- `@secure()` hides parameter values from logs and history
- Azure Verified Modules provide maintained, reusable modules
- The Bicep VS Code extension gives IntelliSense from Azure APIs

MAIN.BICEP

```
@allowed(['dev', 'prod'])
param env string = 'dev'
param location string = resourceGroup().location

var name = 'stshop${env}${uniqueString(resourceGroup().id)}'

resource stg 'Microsoft.Storage/storageAccounts@2024-01-01' = {
  name: take(name, 24)
  location: location
  sku: { name: env == 'prod' ? 'Standard_ZRS' : 'Standard_LRS' }
  kind: 'StorageV2'
  properties: {
    allowBlobPublicAccess: false
    allowSharedKeyAccess: false
    minimumTlsVersion: 'TLS1_2'
  }
}

module kv 'br/public:avm/res/key-vault/vault:0.11.0' = {
  name: 'kv'
  params: { name: 'kv-shop-${env}', enablePurgeProtection: true }
}

output storageId string = stg.id
```

DEPLOY

```
az bicep build -f main.bicep                          # compile / lint
az deployment group what-if -g rg-shop-dev -f main.bicep -p env=dev
az deployment group create -g rg-shop-dev -f main.bicep -p env=dev
az stack group create -n shop -g rg-shop-prod -f main.bicep -p env=prod \
    --action-on-unmanage deleteResources --deny-settings-mode denyDelete
```

CODE EXPLAINED

- → `uniqueString` builds a stable, unique storage name
- → The ternary picks ZRS for prod
- → The module pulls an Azure Verified Module from the public registry
- → Stacks delete removed resources and deny manual deletes

AWS / GCP EQUIVALENT

- ↔ **AWS** CloudFormation templates, change sets, stacks (aws-34)
- ↔ **GCP** Terraform on Google Cloud, Infrastructure Manager (gcp-34, gcp-35)
- ↔ Bicep what-if ≈ CloudFormation change sets ≈ terraform plan

COST CHECK

- $ Bicep and ARM deployments are free
- $ what-if catches accidental SKU upgrades before they bill
- $ Stacks can clean up resources removed from the template, avoiding orphans

SECURITY CHECK

- ⚠ Mark secrets `@secure()` and never output them
- ⚠ Read secrets from Key Vault with `getSecret()` in module parameters
- ⚠ Deny settings on deployment stacks block out-of-band deletes

LIMITS & QUOTAS

- ≤ 800 resources per deployment (including copies)
- ≤ 256 parameters, 256 variables, 64 outputs per template
- ≤ Compiled template up to 4 MB

SCOPES

COMMAND	SCOPE
`az deployment group create`	Resource group
`az deployment sub create`	Subscription
`az deployment mg create`	Management group
`az deployment tenant create`	Tenant
`az stack group / sub / mg create`	Deployment stack

SYNTAX

ELEMENT	EXAMPLE
`Parameter`	param env string = 'dev'
`Variable`	var n = 'st${env}'
`Resource`	resource s 'Type@ver' = {…}
`Module`	module m './m.bicep' = {…}
`Loop`	[for i in range(0, 3): {…}]
`Condition`	resource x '…' = if (env == 'prod') {…}
`Output`	output id string = s.id

DO & DON'T

- ✓ **Do** Run what-if in pull requests
- ✓ **Do** Use Azure Verified Modules for common resources
- ✗ **Don't** Hard-code secrets in parameter files
- ✗ **Don't** Deploy with `--mode Complete` without understanding it deletes unlisted resources

LOCAL TESTING

- △ `az bicep build` and the linter run offline
- △ PSRule for Azure checks templates against best practices
- △ Bicep VS Code extension visualizes resource graphs

PRODUCTION CHECKLIST

- □ Deployments from CI with OIDC (sheet 39)
- □ what-if output reviewed before apply
- □ Deployment stacks with deny settings for production resource groups

Azure Developer CLI (azd)

FROM CODE TO RUNNING APP IN ONE COMMAND

The **Azure Developer CLI (azd)** ties application code to infrastructure (Bicep or Terraform in `infra/`) through an `azure.yaml` file. `azd up` provisions resources and deploys services; `azd deploy` ships code only. **Environments** hold per-stage settings, and `azd pipeline config` sets up GitHub Actions or Azure Pipelines with federated credentials.

KEY POINTS

- Start from a template: `azd init -t <template>` or the Awesome azd gallery
- Service hosts: containerapp, appservice, function, staticwebapp, aks
- Environment values live in `.azure/<env>/.env`
- `azd down --purge` deletes everything including soft-deleted vaults

EVERYDAY AZD

```
azd auth login
azd init -t todo-python-mongo-aca        # or: azd init (detect existing code)
azd env new shop-dev
azd env set AZURE_LOCATION eastus2
azd up                                   # provision + deploy
azd deploy api                           # redeploy one service
azd monitor --overview                   # open Application Insights
azd env new shop-prod && azd up          # second environment
azd pipeline config --provider github    # CI/CD with OIDC
azd down --purge                         # remove everything
```

CODE EXPLAINED

- → A template brings app code, infra and the azure.yaml together
- → Each environment gets its own resource group and settings
- → `azd deploy api` updates only that service
- → `pipeline config` creates the identity, federated credential and workflow

AZURE.YAML

```
name: shop
metadata: { template: shop@1.0.0 }
services:
  api:
    project: ./src/api
    language: python
    host: containerapp
  web:
    project: ./src/web
    language: js
    host: staticwebapp
    dist: dist
infra: { provider: bicep, path: infra }
```

AWS / GCP EQUIVALENT

↔	**AWS**	AWS Copilot CLI / SAM / Amplify (aws-35, aws-36)
↔	**GCP**	Cloud Run `--source` deploys, Cloud Code (gcp-20)

COST CHECK

- $ azd is free; you pay for what the template provisions
- $ Check the template's SKUs before `azd up`: samples may include premium tiers
- $ `azd down --purge` avoids paying for forgotten resources

COMMANDS

COMMAND	DOES
`azd init`	Start from template or code
`azd up`	Provision + deploy
`azd provision`	Infra only
`azd deploy`	Code only
`azd env list / new / select`	Environments
`azd show`	Endpoints
`azd down`	Tear down

PROJECT LAYOUT

PATH	HOLDS
`azure.yaml`	Services and hosts
`infra/main.bicep`	Infrastructure
`src/<service>`	App code
`.azure/<env>/.env`	Environment values (gitignored)
`.github/workflows`	Pipeline from azd

DO & DON'T

- ✓ **Do** Use separate azd environments per stage
- ✓ **Do** Review `infra/` before the first `azd up`
- ✗ **Don't** Commit `.azure/` folders with environment secrets
- ✗ **Don't** Use azd in production without pinning template versions

LOCAL TESTING

- ⌂ `azd` runs locally against your subscription; app services can run locally with Docker / `func start`
- ⌂ Dev containers in many templates set up the toolchain
- ⌂ `azd provision --preview` shows planned changes

PRODUCTION CHECKLIST

- ☐ Production environment deployed only by the generated pipeline
- ☐ Template infra reviewed and SKUs right-sized
- ☐ Environment settings stored in Key Vault / pipeline variables

Terraform on Azure

`azurerm` & `azapi` providers · state in a storage account · OIDC authentication · `terraform plan` / `apply` · choosing Bicep vs azd vs Terraform

TERRAFORM WITH AZURE

The **azurerm** provider covers most Azure services with typed resources; **azapi** calls any ARM API version directly, useful for brand-new features. State lives in a storage account container with blob leases for locking. In CI, authenticate with **OIDC** (federated credentials) instead of client secrets.

KEY POINTS

- azurerm 4.x requires `subscription_id` in the provider block
- Use `use_azuread_auth = true` for the state backend
- Import existing resources with `import` blocks
- Azure Verified Modules also exist for Terraform

PROVIDER, BACKEND, RESOURCES

```hcl
terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
    azapi   = { source = "Azure/azapi", version = "~> 2.0" }
  }
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "sttfstateshop"
    container_name       = "tfstate"
    key                  = "shop/prod.tfstate"
    use_azuread_auth     = true
    use_oidc             = true
  }
}

provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
  use_oidc        = true
}

resource "azurerm_resource_group" "shop" {
  name     = "rg-shop-prod"
  location = "eastus2"
  tags     = { owner = "team-a", env = "prod" }
}

resource "azurerm_container_app_environment" "cae" {
  name                = "cae-shop-prod"
  resource_group_name = azurerm_resource_group.shop.name
  location            = azurerm_resource_group.shop.location
}
```

CODE EXPLAINED

→ Version constraints keep provider upgrades deliberate

→ The backend stores state in a blob and locks with leases

→ `use_oidc` reads the GitHub OIDC token in CI

→ Resources reference each other to build dependencies

AWS / GCP EQUIVALENT

↔ **AWS** Terraform AWS provider, S3 state (aws-36)

↔ **GCP** Terraform Google provider, Cloud Storage state (gcp-34)

COST CHECK

$ Terraform CLI is free; HCP Terraform has free and paid tiers

$ State storage costs pennies

$ `terraform plan` in PRs catches costly changes (pair with Infracost)

CHOOSING AN IAC TOOL

TOOL	BEST WHEN
Bicep	Azure only, day-one API support
azd + Bicep	App + infra templates, dev loop
Terraform azurerm	Multi-cloud teams, existing Terraform
azapi	Features azurerm lacks yet
Pulumi	General-purpose languages

COMMANDS

COMMAND	DOES
`terraform init`	Providers + backend
`terraform plan -out p`	Preview
`terraform apply p`	Apply saved plan
`terraform import / import {}`	Adopt resources
`terraform state list`	Managed resources

DO & DON'T

✓ **Do** Lock provider versions and upgrade on purpose

✓ **Do** One state file per environment and component

✗ **Don't** Store state locally or in git

✗ **Don't** Mix manual portal changes with Terraform-managed resources

LOCAL TESTING

⌂ `terraform validate` / `fmt` / `tflint --enable-plugin=azurerm` run offline

⌂ `terraform plan` against a sandbox subscription

⌂ Checkov / Trivy scan for misconfigurations

PRODUCTION CHECKLIST

- ☐ Apply only from CI with OIDC and approvals
- ☐ State account with versioning, soft delete and private access
- ☐ Drift detection on a schedule

Azure Monitor Metrics & Alerts

WATCHING RESOURCES

Azure Monitor collects **platform metrics** for every resource automatically (CPU, requests, latency). **Diagnostic settings** send resource logs and metrics to Log Analytics, storage or Event Hubs. **Alert rules** fire on metric thresholds, log queries or activity log events, and notify **action groups** (email, SMS, webhooks, Functions, ITSM). **Workbooks** build interactive reports.

KEY POINTS

- Platform metrics are kept 93 days
- Dynamic thresholds learn normal patterns
- Alert processing rules suppress alerts during maintenance
- Azure Monitor Baseline Alerts (AMBA) give recommended alert sets

METRICS, ACTION GROUPS, ALERTS

```
VM=$(az vm show -g rg-app -n vm-web-1 --query id -o tsv)
az monitor metrics list --resource $VM --metric "Percentage CPU" \
    --interval PT5M --aggregation Average -o table

az monitor action-group create -g rg-ops -n ag-oncall --short-name oncall \
    --action email oncall ops@contoso.com \
    --action webhook teams https://contoso.webhook.office.com/...

az monitor metrics alert create -g rg-ops -n cpu-high --scopes $VM \
    --condition "avg Percentage CPU > 80" --window-size 5m \
    --evaluation-frequency 1m --severity 2 --action ag-oncall

# Send a resource's logs to Log Analytics
az monitor diagnostic-settings create -n to-law --resource $KV \
    --workspace $LAW --logs '[{"categoryGroup":"allLogs","enabled":true}]' \
    --metrics '[{"category":"AllMetrics","enabled":true}]'

az monitor activity-log alert create -g rg-ops -n rg-deleted \
    --condition category=Administrative and \
      operationName=Microsoft.Resources/subscriptions/resourceGroups/delete \
    --action-group ag-oncall
```

CODE EXPLAINED

- → `metrics list` pulls data points for a time grain
- → One action group can notify many channels
- → The alert evaluates every minute over a 5-minute window
- → Activity log alerts fire on management events such as deletes

AWS / GCP EQUIVALENT

- ↔ **AWS** CloudWatch metrics, alarms, SNS actions (aws-37)
- ↔ **GCP** Cloud Monitoring metrics, alerting policies, notification channels (gcp-37)

COST CHECK

- $ Platform metrics are free; metric alert rules bill per time series monitored
- $ Log ingestion via diagnostic settings bills per GB
- $ Send only needed log categories; use Basic / Auxiliary tables for verbose logs

ALERT TYPES

TYPE	WATCHES
`Metric`	Thresholds / dynamic
`Log search`	KQL query results
`Activity log`	Management events
`Service health`	Azure incidents
`Resource health`	One resource's availability
`Smart detection`	App Insights anomalies

SEVERITIES

SEV	MEANING
`0`	Critical
`1`	Error
`2`	Warning
`3`	Informational
`4`	Verbose

DO & DON'T

- ✓ **Do** Alert on user-impacting symptoms with runbook links
- ✓ **Do** Use alert processing rules for maintenance windows
- ✗ **Don't** Alert on every metric at Sev 0
- ✗ **Don't** Send all log categories from every resource without a plan

LOCAL TESTING

- ⌂ Query metrics with the CLI against a dev subscription
- ⌂ Test webhooks with a request bin before wiring production channels
- ⌂ Workbooks can be authored in a sandbox and exported as JSON

PRODUCTION CHECKLIST

- ☐ Baseline alerts (AMBA) deployed for core services
- ☐ Action groups with on-call rotation integration
- ☐ Service health and resource health alerts enabled

Sheet 38

Log Analytics & Application Insights

workspaces · KQL queries · retention & table plans · Application Insights with the OpenTelemetry distro · distributed tracing · `az monitor log-analytics query`

LOGS, TRACES AND KQL

A **Log Analytics workspace** stores logs in tables queried with **KQL** (Kusto Query Language). **Application Insights** (workspace-based) collects app requests, dependencies, exceptions and traces; instrument with the **Azure Monitor OpenTelemetry distro**. Table plans (Analytics, Basic, Auxiliary) and retention settings control cost.

KEY POINTS

- App Insights tables: `AppRequests`, `AppDependencies`, `AppExceptions`, `AppTraces`
- Operation IDs join a request with its dependencies across services
- Interactive retention up to 2 years; long-term retention up to 12 years
- One workspace per environment or region is a common pattern

INSTRUMENT A PYTHON APP

```python
from azure.monitor.opentelemetry import configure_azure_monitor
from opentelemetry import trace
configure_azure_monitor()   # reads APPLICATIONINSIGHTS_CONNECTION_STRING
tracer = trace.get_tracer(__name__)

def checkout(cart):
    with tracer.start_as_current_span("checkout") as span:
        span.set_attribute("cart.items", len(cart))
        return charge(cart)
```

CODE EXPLAINED

- → One call wires traces, metrics and logs to Application Insights
- → Custom spans and attributes become searchable
- → `summarize` with `percentile` gives p95 latency per operation
- → The App Insights component points at a workspace

KQL QUERIES

```
AppRequests
| where TimeGenerated > ago(1h)
| summarize count(), p95 = percentile(DurationMs, 95) by Name, ResultCode
| order by p95 desc

AppExceptions
| where TimeGenerated > ago(24h)
| summarize n = count() by ProblemId
| top 10 by n
```

AWS / GCP EQUIVALENT

- ↔ **AWS** CloudWatch Logs, Logs Insights, X-Ray / ADOT (aws-38)
- ↔ **GCP** Cloud Logging, Log Analytics (SQL), Cloud Trace (gcp-38)

CREATE AND QUERY

```
az monitor log-analytics workspace create -g rg-ops -n law-shop-prod \
    --retention-time 90
az monitor app-insights component create -g rg-ops -a ai-shop-prod \
    -l eastus2 --workspace law-shop-prod
az monitor log-analytics query -w <workspace-guid> \
    --analytics-query "AppRequests | take 5" -o table
```

COST CHECK

- $ Analytics logs bill per GB ingested; Basic and Auxiliary plans are cheaper but limited
- $ Commitment tiers discount high daily volumes
- $ Sampling in App Insights cuts telemetry volume and cost

LIMITS & QUOTAS

- ≤ Query results up to 500,000 rows or 64 MB
- ≤ Query timeout 10 minutes
- ≤ Daily cap optional; ingestion stops when reached

KQL BASICS

OPERATOR	DOES
`where`	Filter rows
`project`	Pick columns
`summarize … by`	Aggregate
`extend`	Add a column
`join`	Combine tables
`render timechart`	Chart in portal
`ago(1h)`	Relative time

TABLE PLANS

PLAN	USE
Analytics	Full KQL, alerts, default
Basic	Cheaper, limited queries
Auxiliary	Low-cost verbose / audit logs

DO & DON'T

- ✓ **Do** Use workspace-based App Insights with the OpenTelemetry distro
- ✓ **Do** Set retention per table to match need
- ✗ **Don't** Log at debug level in production without sampling
- ✗ **Don't** Create a workspace per resource

LOCAL TESTING

- ⌂ The OpenTelemetry console exporter prints spans during development
- ⌂ Run queries in the portal's Logs blade or Azure Data Explorer web UI against the workspace
- ⌂ Use the Kusto VS Code extension for KQL authoring

PRODUCTION CHECKLIST

- ☐ Sampling and daily cap tuned
- ☐ Alerts on failures and p95 latency from log queries
- ☐ Retention and table plans set for cost and compliance

Deploying to Azure

GitHub Actions `azure/login` with OIDC · Azure Pipelines YAML · slot swaps · Container Apps revision rollouts · environments & approvals · tagging strategy

CI/CD INTO AZURE

Pipelines sign in with **OIDC**: GitHub Actions uses `azure/login` with a user-assigned identity or app registration that has a **federated credential**; Azure Pipelines uses a **workload identity federation** service connection. Deploy with slot swaps (App Service), revision traffic shifts (Container Apps) or IaC deployments, behind **environment** approvals.

KEY POINTS

- No client secrets or publish profiles in CI
- Give the deploy identity only the roles it needs on target resource groups
- Build once, deploy the same artifact to each environment
- Tag resources with repo and commit for traceability

GITHUB ACTIONS: BUILD, PUSH, DEPLOY TO CONTAINER APPS

```yaml
on: { push: { branches: [main] } }
permissions: { id-token: write, contents: read }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: az acr build -r shopacr -t api:${{ github.sha }} .
      - run: |
          az containerapp update -g rg-app -n api \
            --image shopacr.azurecr.io/api:${{ github.sha }} \
            --tags commit=${{ github.sha }}
```

CODE EXPLAINED

- → `id-token: write` is required for OIDC
- → The client ID belongs to the identity with the federated credential
- → `az acr build` builds in Azure, so the runner needs no Docker login
- → The service connection uses federation instead of a secret

AWS / GCP EQUIVALENT

- ↔ **AWS** configure-aws-credentials OIDC, CodeDeploy (aws-39)
- ↔ **GCP** google-github-actions/auth, Cloud Deploy (gcp-39)

AZURE PIPELINES STEP

```yaml
- task: AzureCLI@2
  inputs:
    azureSubscription: sc-shop-prod      # workload identity federation
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: az deployment group create -g rg-shop-prod -f infra/main.bicep
```

COST CHECK

- $ GitHub-hosted minutes and Azure Pipelines parallel jobs have free tiers
- $ ACR Tasks builds bill per second
- $ Staging slots and revisions reuse existing capacity

SECURITY CHECK

- ⚠ Federated subjects bound to protected environments (`environment:production`)
- ⚠ Deploy identity scoped to specific resource groups, not the subscription
- ⚠ Disable basic auth publishing credentials on App Service

DEPLOY METHODS

TARGET	METHOD
App Service	webapps-deploy / zip deploy + slot swap
Container Apps	az containerapp update / revisions
Functions	functions-action / func publish
AKS	kubectl / Helm / Flux GitOps
Infrastructure	az deployment / azd / Terraform

PIPELINE IDENTITY

OPTION	VERDICT
Federated credential (OIDC)	Use
Workload identity service connection	Use (Azure Pipelines)
Client secret	Avoid
Publish profile	Avoid; disable basic auth

DO & DON'T

- ✓ **Do** Gate production with environment approvals
- ✓ **Do** Promote one immutable image by digest or SHA tag
- ✗ **Don't** Use publish profiles or client secrets
- ✗ **Don't** Give the pipeline Owner on the subscription

LOCAL TESTING

- ⌂ `act` runs workflows locally (OIDC steps need GitHub)
- ⌂ actionlint validates workflow syntax
- ⌂ Run the same `az` commands from a laptop against a dev environment first

PRODUCTION CHECKLIST

- ☐ OIDC-only pipeline identity with least privilege
- ☐ Approvals and change records for production
- ☐ Rollback documented (slot swap back / revision traffic shift)

Microsoft Foundry & Azure OpenAI

Foundry resources & projects · model deployments · Responses / chat completions API · Entra ID auth over API keys · content safety · Foundry Agent Service · RAG with AI Search

GENERATIVE AI ON AZURE

Microsoft Foundry (formerly Azure AI Foundry) is Azure's platform for models, agents and evaluations. You create a Foundry resource and **projects**, **deploy** models (OpenAI GPT models plus others from the catalog) with a deployment name and quota, then call them through the OpenAI-compatible **v1 API** (Responses or chat completions) using Entra ID. **Content filters** run by default; **Foundry Agent Service** builds tool-using agents.

KEY POINTS

- Deployment types: Global Standard, Data Zone, Standard (regional), Provisioned
- Your code calls the deployment name, not the raw model name
- Use the OpenAI SDK with an Azure base URL and Entra token provider
- RAG: retrieve from Azure AI Search, then pass results to the model

CREATE A RESOURCE AND DEPLOYMENT

```
az cognitiveservices account create -g rg-ai -n ai-shop -l eastus2 \
    --kind AIServices --sku S0 --custom-domain ai-shop \
    --disable-local-auth true
az cognitiveservices account deployment create -g rg-ai -n ai-shop \
    --deployment-name chat --model-name gpt-5-mini \
    --model-version <version> --model-format OpenAI \
    --sku-name GlobalStandard --sku-capacity 50
AI=$(az cognitiveservices account show -g rg-ai -n ai-shop --query id -o tsv)
az role assignment create --assignee-object-id $MI \
    --assignee-principal-type ServicePrincipal \
    --role "Cognitive Services OpenAI User" --scope $AI
```

CODE EXPLAINED

- → `--disable-local-auth` removes API keys from the resource
- → `sku-capacity 50` sets thousands of tokens per minute for the deployment
- → The token provider refreshes Entra tokens for the OpenAI client
- → `model` is the deployment name you chose

AWS / GCP EQUIVALENT

- ↔ **AWS** Amazon Bedrock (InvokeModel / Converse, Agents, Guardrails) (aws-40)
- ↔ **GCP** Agent Platform (Vertex AI) with Gemini (gcp-40)

CALL IT (PYTHON, ENTRA ID)

```
from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
token = get_bearer_token_provider(
    DefaultAzureCredential(), "https://cognitiveservices.azure.com/.default")
client = OpenAI(base_url="https://ai-shop.openai.azure.com/openai/v1/",
                api_key=token)
resp = client.responses.create(
    model="chat",                                  # deployment name
    input="Summarize our return policy in 3 bullets.")
print(resp.output_text)
```

COST CHECK

- $ Pay per input / output token on Standard deployments
- $ Provisioned throughput: reserved capacity, hourly or monthly
- $ Smaller models (mini / nano) cost a fraction for simple tasks

SECURITY CHECK

- ⚠ Disable key-based auth; grant Cognitive Services OpenAI User to app identities
- ⚠ Private endpoints for the Foundry resource
- ⚠ Keep content filters on and log prompts responsibly

LIMITS & QUOTAS

- ≤ Quota is tokens per minute (TPM) per model per region per subscription
- ≤ Requests per minute follow from TPM; exceeding returns 429 with retry-after
- ≤ Request quota increases in the Foundry portal

DEPLOYMENT TYPES

TYPE	USE
Global Standard	Highest quota, data may process in any region
Data Zone Standard	Processing within a geography (US / EU)
Standard (regional)	Single region
Provisioned (PTU)	Reserved, predictable latency
Batch	Async jobs at lower price

FOUNDRY PIECES

PIECE	DOES
Foundry resource	Hosts models and projects
Project	Workspace for an app or team
Model deployment	Named endpoint with quota
Agent Service	Agents with tools and threads
Evaluations	Quality and safety tests
Content Safety	Filters and shields

DO & DON'T

- ✓ **Do** Use Entra ID auth and managed identity
- ✓ **Do** Handle 429s with retry-after and backoff
- ✗ **Don't** Ship API keys in client apps
- ✗ **Don't** Send sensitive data without checking data residency options

LOCAL TESTING

- ⌂ Call a dev deployment from a notebook with `az login` credentials
- ⌂ Prompt and evaluation work in the Foundry portal playground
- ⌂ Mock the OpenAI client in unit tests; test prompts with evals

PRODUCTION CHECKLIST

- □ Key auth disabled, private endpoint, content filters on
- □ Quota monitored with alerts on 429 rates
- □ Evaluations run on prompt or model changes

Foundry Tools & ML

Foundry Tools (formerly Azure AI Services): Vision, Speech, Language, Translator, Document Intelligence · Azure Machine Learning · GitHub Copilot pointer

PREBUILT AI AND CUSTOM ML

Foundry Tools (formerly Azure AI Services) are prebuilt APIs: **Vision** (image analysis, OCR), **Speech** (speech-to-text, text-to-speech, translation), **Language** (sentiment, entities, PII detection, summarization), **Translator** and **Document Intelligence** (forms, invoices, receipts). **Azure Machine Learning** trains, tracks and deploys your own models.

KEY POINTS

- One multi-service resource can serve several tools with one endpoint
- Prebuilt models need no training; custom models train on your labeled data
- Containers exist for some tools to run on-premises or at the edge
- GitHub Copilot is the developer assistant (separate product)

DOCUMENT INTELLIGENCE: READ AN INVOICE

```python
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.identity import DefaultAzureCredential
client = DocumentIntelligenceClient(
    "https://ai-shop.cognitiveservices.azure.com/", DefaultAzureCredential())
with open("invoice.pdf", "rb") as f:
    poller = client.begin_analyze_document("prebuilt-invoice", body=f)
doc = poller.result().documents[0]
print(doc.fields["VendorName"].value_string, doc.fields["InvoiceTotal"].content)
```

CODE EXPLAINED

- → `prebuilt-invoice` extracts vendor, totals and line items
- → Long operations return a poller; `result()` waits
- → PII detection returns redacted text and the entities found
- → Azure ML jobs are defined in YAML and run on managed compute

LANGUAGE: DETECT PII

```python
from azure.ai.textanalytics import TextAnalyticsClient
from azure.identity import DefaultAzureCredential
ta = TextAnalyticsClient("https://ai-shop.cognitiveservices.azure.com/",
                         DefaultAzureCredential())
res = ta.recognize_pii_entities(["Call Ada at 555-0100 about order 42."])[0]
print(res.redacted_text)        # Call *** at ******** about order 42.
```

AWS / GCP EQUIVALENT

- ↔ **AWS** Rekognition, Textract, Transcribe, Comprehend, SageMaker AI (aws-41)
- ↔ **GCP** Vision API, Document AI, Speech-to-Text, Vertex AI training (gcp-41)

AZURE MACHINE LEARNING CLI

```bash
az extension add -n ml
az ml workspace create -g rg-ml -n mlw-shop
az ml job create -f train-job.yml -g rg-ml -w mlw-shop
```

COST CHECK

- $ Per transaction / page / audio hour, with free monthly tiers
- $ Commitment tiers discount high volumes
- $ Azure ML: you pay for compute while jobs and endpoints run

TOOLS

TOOL	DOES
Vision	Image analysis, OCR
Speech	STT, TTS, speech translation
Language	Sentiment, PII, entities, summarization
Translator	Text and document translation
Document Intelligence	Forms, invoices, receipts, IDs
Content Safety	Harmful content detection
Azure ML	Train / deploy custom models

DO & DON'T

- ✓ **Do** Start with prebuilt models before training custom ones
- ✓ **Do** Use Entra ID auth and regional endpoints that meet data rules
- ✗ **Don't** Send documents with personal data without reviewing retention settings
- ✗ **Don't** Call synchronous APIs in a loop for large batches (use batch APIs)

LOCAL TESTING

- ⌂ Several tools ship Docker containers (e.g. Language, Speech) for local or disconnected use
- ⌂ Foundry portal playgrounds test tools without code
- ⌂ Record sample responses and replay them in unit tests

PRODUCTION CHECKLIST

- ☐ Managed identity auth and private endpoints
- ☐ Retries for 429 / transient errors
- ☐ Monitoring of usage against quotas and budgets

Sheet 42

Quick Reference — az Commands & Service Picker

az group · vm · storage · ad / role · functionapp / webapp · acr / containerapp / aks · monitor · deployment · Need → Azure service → AWS → Google Cloud (with sheet numbers) · default limits worth knowing · pricing model per service

ACCOUNT & GROUPS

COMMAND	DOES	SHEET #
az login / az account set -s SUB	Sign in, pick subscription	03
az group create -n RG -l LOC	Resource group	01
az group delete -n RG --yes --no-wait	Delete everything in RG	01
az resource list -g RG -o table	List resources	01
az vm list-usage -l LOC	Quota usage	15

IDENTITY

COMMAND	DOES	SHEET #
az ad signed-in-user show	Who am I	05
az role assignment create	Grant a role	06
az role assignment list --assignee X --all	Audit grants	06
az identity create	User-assigned identity	07
az keyvault secret set / show	Secrets	08

COMPUTE & APPS

COMMAND
az vm create / deallocate / start
az vmss create
az webapp up / deploy
az webapp deployment slot swap
az functionapp create
func start / func azure functionapp publish

CONTAINERS

COMMAND	DOES	S #
az acr login / build	Registry	2
az containerapp up / update	Container Apps	2
az containerapp ingress traffic set	Canary split	2
az aks create / get-credentials	AKS	2
az aks nodepool add	Node pool	2

DATA, IAC & OPS

COMMAND	DOES
az storage blob upload --auth-mode login	Upload blob
az sql db create / az postgres flexible-server create	Databases
az deployment group what-if / create	Bicep deploy
azd up / deploy / down	Developer CLI
az monitor metrics alert create	Alerts
az monitor log-analytics query	KQL

SERVICE PICKER

NEED	AZURE	AWS	GOOGLE CLOUD	SHEET #
Identity directory	Entra ID	IAM Identity Center	Cloud Identity	05
Workload identity	Managed identity	IAM role	Service account	07
Secrets	Key Vault	Secrets Manager	Secret Manager	08
Private network	VNet	VPC	VPC (global)	10
Global L7 + CDN	Front Door	CloudFront	Global ALB + Cloud CDN	14
VMs / scale	VMs / VMSS	EC2 / ASG	Compute Engine / MIG	15, 18
Functions	Azure Functions	Lambda	Cloud Run functions	20
Serverless containers	Container Apps	ECS Fargate / App Runner	Cloud Run	26
Kubernetes	AKS	EKS	GKE	27
Object storage	Blob Storage	S3	Cloud Storage	28
Relational	Azure SQL / PostgreSQL Flexible	RDS / Aurora	Cloud SQL / AlloyDB	31
NoSQL	Cosmos DB	DynamoDB	Firestore	32
Queue / topic	Service Bus	SQS / SNS	Pub/Sub	24
Native IaC	Bicep	CloudFormation / CDK	Infrastructure Manager	34
Logs query	Log Analytics (KQL)	CloudWatch Logs Insights	Cloud Logging	38
Generative AI	Microsoft Foundry	Bedrock	Agent Platform (Vertex AI)	40

LIMITS WORTH KNOWING

LIMIT	DEFAULT
Resource groups per subscription	980
Role assignments per subscription	4,000
Subnets per VNet	3,000
Rules per NSG	1,000
Functions HTTP response	~230 s
Service Bus message (Standard)	256 KB
Cosmos DB item	2 MB
ARM template resources	800

PRICING MODELS

SERVICE	BILLED BY
VMs	Per minute running + disks
App Service	Per plan instance-hour
Functions Flex / Container Apps	Per vCPU / GiB-second
Blob Storage	GB-month + operations + egress
Cosmos DB	RU/s (or RUs) + storage
Log Analytics	GB ingested + retention
Foundry models	Tokens or provisioned units